A PRELIMINARY REPORT ON

WATERMARK-GUIDED TAMPER LOCALIZATION AND RECOVERY

A Key-Seeded Self-Embedding Fragile Watermarking Scheme with Joint Localization and Restoration Assessment

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING
(COMPUTER ENGINEERING)

SUBMITTED BY

[STUDENT NAME 1]	**Exam No : [EXAM NO.]**
[STUDENT NAME 2]	**Exam No : [EXAM NO.]**
[STUDENT NAME 3]	**Exam No : [EXAM NO.]**
[STUDENT NAME 4]	**Exam No : [EXAM NO.]**

Under the Guidance of

Prof. [Guide Name]

DEPARTMENT OF COMPUTER ENGINEERING
AJEENKYA DY PATIL SCHOOL OF ENGINEERING
CHARHOLI BUDRUK, LOHEGAON, PUNE 412105

SAVITRIBAI PHULE PUNE UNIVERSITY
2026 - 2027

AJEENKYA DY PATIL SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the Project Report entitled

"WATERMARK-GUIDED TAMPER LOCALIZATION AND RECOVERY"

Submitted by

[STUDENT NAME 1]	**Exam No : [EXAM NO.]**
[STUDENT NAME 2]	**Exam No : [EXAM NO.]**
[STUDENT NAME 3]	**Exam No : [EXAM NO.]**
[STUDENT NAME 4]	**Exam No : [EXAM NO.]**

are bonafide students of this institute and the work has been carried out by them under the supervision of Prof. [Guide Name] and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of Bachelor of Engineering in Computer Engineering.

(Prof. [Guide Name])	(Dr. Pankaj Agarkar)
Guide	H.O.D
Department of Computer Engineering	Department of Computer Engineering

Signature of Internal Examiner:

Signature of External Examiner:

Dr. F. B. Sayyad
Principal
AJEENKYA DY PATIL SCHOOL OF ENGINEERING, Lohegaon, Pune-412105

Place : Pune

Date :

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the preliminary project report on "Watermark-Guided Tamper Localization And Recovery".

We would like to take this opportunity to thank our Internal Guide Prof. [Guide Name] for giving us all the help and guidance we needed. We are really grateful for the kind support, and the valuable suggestions that shaped both the design of the scheme and the way its results are reported.

We are also grateful to Dr. Pankaj Agarkar, Head of the Computer Engineering Department, ADYPSOE, Lohegaon, Pune, and to our Project Co-ordinator Dr. Amruta Chitari, for their indispensable support, suggestions and motivation during the entire course of the project.

We would also be grateful to our Principal Dr. F. B. Sayyad, who encouraged us and created a healthy environment for all of us to learn in the best possible way.

We also thank all the staff members of our college and the technicians for their help in making this project a successful one.

[STUDENT NAME 1]
[STUDENT NAME 2]
[STUDENT NAME 3]
[STUDENT NAME 4]

ABSTRACT

Region-level image manipulation is now free, fast and visually seamless, and the lossless formats in which evidence is normally stored, such as PNG, BMP and TIFF, protect nothing about content. Passive detectors, which analyze only the received image, answer half of the question at best: they may suggest that a region looks altered, but they cannot state which blocks were touched, cannot say what the region contained, and return no verdict that a third party can repeat. The problem addressed here is to localize tampering at block granularity and restore what was destroyed, for images that pass through a lossless custody chain.

This project presents a deterministic, training-free self-embedding fragile watermarking scheme. The image is divided into 8x8 blocks, and the two least-significant bit-planes of each block carry a 128-bit payload: a 32-bit keyed HMAC-SHA256 tag computed over the block's own six most-significant bit-planes, and a 96-bit recovery descriptor of a spatially distant partner block chosen by a key-seeded permutation. Verification recomputes each tag, and a mismatch flags the block. A flagged block is rebuilt from the descriptor its surviving partner carries; where that partner is also flagged, the block is marked unrecoverable rather than fabricated.

Across a 32-image corpus, embedding reaches a mean PSNR of 45.479 dB and SSIM of 0.98847. On the 1,792-trial main grid, block-level recall is 1.0000 for splicing, crop-and-refill and noise corruption, and 0.99998 for inpainting removal. The null condition, 1,802,240 verifications of unmodified images, produced no false-positive block. The recoverability rate sits 3 to 6 percentage points above its structural bound of 1 - alpha at every tamper ratio, falling from 0.9590 at 10% tamper to 0.3579 at 70%, and in-region recovery quality is 28.49 dB for Variant A and 31.13 dB for Variant C. What distinguishes the design is accountability rather than accuracy: every decision is a single recomputed keyed hash that any third party holding the key can repeat bit for bit.

Keywords: fragile watermarking; self-embedding; tamper localization; image recovery; HMAC-SHA256; image forensics; content authentication; recoverability rate

TABLE OF CONTENTS

ACKNOWLEDGEMENT ..... **i**
ABSTRACT ..... **ii**
LIST OF ABBREVIATIONS ..... **v**
LIST OF FIGURES ..... **vi**
LIST OF TABLES ..... **vii**

CHAPTER 1: INTRODUCTION **1**
1.1 Motivation of the Project 1
1.2 Literature Survey 3
1.2.1 Passive and Active Authentication 3
1.2.2 Self-Embedding and Its Known Failure Modes 3
1.2.3 Learned Proactive Watermarking 4
1.2.4 Market Survey and Research Gap 5
1.3 Organization of the Report 6
CHAPTER 2: PROBLEM DEFINITION AND SCOPE **7**
2.1 Problem Statement 7
2.2 Goal and Objectives 7
2.3 Statement of Scope 8
2.4 Methodologies of Problem Solving and Efficiency Issues 10
CHAPTER 3: PROJECT PLAN **12**
3.1 Project Estimates 12
3.2 Risk Management with respect to NP-Hard Analysis 13
3.3 Project Schedule 15
CHAPTER 4: SOFTWARE REQUIREMENTS SPECIFICATION **18**
4.1 Database Requirements 18
4.2 Software Requirements (Platform Choice) 19
4.3 Hardware Requirements 20
4.4 Analysis Models: SDLC Model to be Applied 21
CHAPTER 5: SYSTEM DESIGN **24**
5.1 System Architecture 24
5.2 Mathematical Model 26
5.3 Data Flow Diagrams 29
5.3.1 Level 0 (Context Diagram) 29
5.3.2 Level 1 30
5.3.3 Level 2 30

5.4 Entity Relationship Diagram.....31
5.5 UML Diagrams.....32
5.5.1 Use Case Diagram.....32
5.5.2 Class Diagram.....33
5.5.3 Sequence Diagram.....34
5.5.4 Activity Diagram.....35
5.5.5 State Machine Diagram.....36
5.5.6 Component Diagram.....37
5.5.7 Deployment Diagram.....37
CHAPTER 6: OTHER SPECIFICATION.....39
6.1 Advantages.....39
6.2 Limitations.....40
6.3 Applications.....41
CHAPTER 7: RESULTS, CONCLUSIONS AND FUTURE WORK.....42
7.1 Results Obtained to Date.....42
7.1.1 Imperceptibility.....42
7.1.2 Localization.....43
7.1.3 Recovery.....43
7.1.4 Comparison and Ablation.....45
7.2 Conclusions.....46
7.3 Future Work.....47
REFERENCES.....49
APPENDIX A: FEASIBILITY ASSESSMENT, SATISFIABILITY ANALYSIS AND COMPLEXITY CLASSIFICATION.....52
APPENDIX B: DETAILS OF THE PAPERS REFERRED.....56
APPENDIX C: PLAGIARISM REPORT.....63

LIST OF ABBREVIATIONS

Abbreviation	Illustration
ADYPSOE	Ajeenkya DY Patil School of Engineering
C2PA	Coalition for Content Provenance and Authenticity
COCOMO	Constructive Cost Model
CPU	Central Processing Unit
DCT	Discrete Cosine Transform
DFD	Data Flow Diagram
DICOM	Digital Imaging and Communications in Medicine
ER	Entity-Relationship
FPR	False Positive Rate
GPU	Graphics Processing Unit
HMAC	Hash-based Message Authentication Code
IoU	Intersection over Union
KLOC	Thousands of Lines of Code
LSB	Least Significant Bit
MSB	Most Significant Bit
PACS	Picture Archiving and Communication System
PNG	Portable Network Graphics
PSNR	Peak Signal-to-Noise Ratio
SDLC	Software Development Life Cycle
SHA	Secure Hash Algorithm
SPPU	Savitribai Phule Pune University
SQL	Structured Query Language
SSIM	Structural Similarity Index Measure
UML	Unified Modeling Language
USC-SIPI	University of Southern California, Signal and Image Processing Institute

LIST OF FIGURES

Figure	Illustration	Page No.
1.1	Passive detection against active self-embedding	2
2.1	The four tamper classes	9
3.1	Project schedule (Gantt chart)	16
4.1	The incremental SDLC model as applied	23
5.1	System architecture	25
5.2	Per-block payload layout at B = 8	27
5.3	Key-seeded block mapping	28
5.4	Data flow diagram, Level 0 (context)	30
5.5	Data flow diagram, Level 1	30
5.6	Data flow diagram, Level 2	31
5.7	Entity-relationship diagram	32
5.8	Use case diagram	33
5.9	Class diagram	34
5.10	Sequence diagram	35
5.11	Activity diagram	36
5.12	State machine diagram	36
5.13	Component diagram	37
5.14	Deployment diagram	38
7.1	Recoverability rate against tamper ratio	44
7.2	In-region and whole-image PSNR	45
7.3	Qualitative result under copy-paste splicing	45

LIST OF TABLES

Table	Illustration	Page No.
1.1	Summary of the surveyed literature	4
2.1	Tamper classes and tamper ratios	9
3.1	Measured size of the implementation	12
3.2	Basic COCOMO estimate	13
3.3	Risk register	14
3.4	Plan of work and status	16
4.1	Schema of the protected-image library	18
4.2	Software requirements	20
4.3	Hardware requirements	21
5.1	Per-block bit budget	26
5.2	Functional modules and responsibilities	25
7.1	Imperceptibility of watermarked images	42
7.2	Recoverability and in-region recovery quality	44
7.3	Null condition on unmodified images	43
7.4	Comparison with published schemes	45
7.5	Block-size ablation	46

CHAPTER 1
INTRODUCTION

1.1 MOTIVATION OF THE PROJECT

The evidentiary value of a digital image rests on an increasingly unsafe assumption: that the pixels presented are the pixels captured. Region-level manipulation, such as pasting an object from one photograph into another, erasing an object with an inpainting tool, or replacing a cropped region with plausible texture, is now available in free software, and diffusion-based editors require neither skill nor time. Everyday formats such as PNG, TIFF and JPEG record pixel values and protect nothing about whether they are the ones the camera or scanner produced.

The stakes are concrete. A radiological study with a lesion erased can change a diagnosis, a vehicle-damage photograph can be edited to inflate or invent an insurance loss, and a surveillance frame entered into a legal record can have a person removed from it. In each case the image is useful because of an integrity the file does not protect, and the party who relies on it usually cannot inspect the original.

Two broad families of countermeasure exist. Passive, or blind, forensics analyses the received image alone for statistical inconsistencies, and its modern instances are learned networks [2], [3]. Active approaches insert authentication information before the image is distributed, so that verification becomes a test of self-consistency rather than an inference about provenance. Fragile watermarking belongs to this second family [4], [5]. The passive detector must infer tampering from evidence the attacker may have concealed; the active scheme checks the image against information the attacker could not forge.

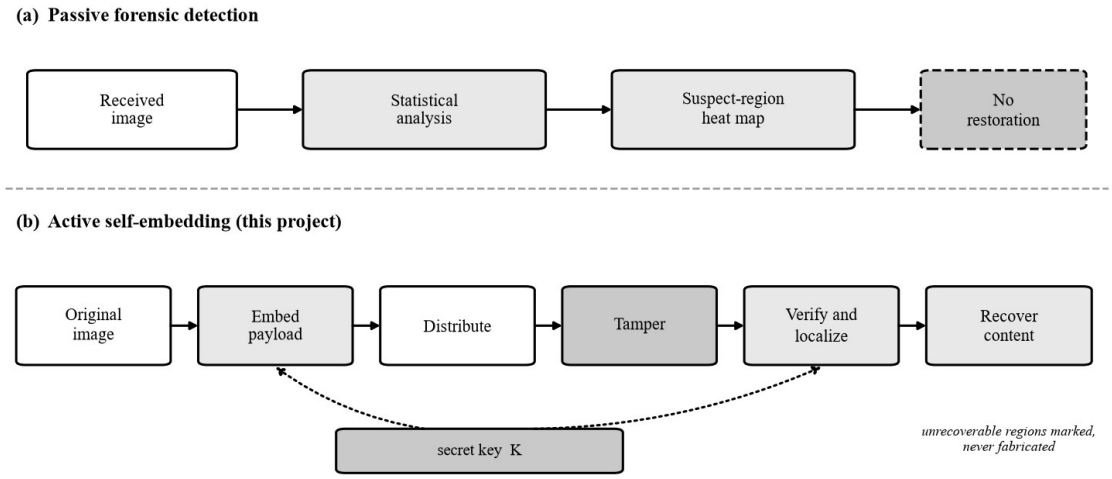


Fig. 1.1: Passive forensic detection against active self-embedding. Only the active path carries the original content forward.

Fig. 1.1 contrasts the two paths. On the passive path, statistical analysis of the received image yields a heat map of suspect regions, with no restoration. On the active path, the original image is embedded with authentication and recovery data, distributed, possibly tampered with, and verified; verification localizes the altered blocks, and recovery rebuilds them from information the image itself carries. The second path is the subject of this project. It ends not at a suspicion but at either a reconstructed block or an explicit statement that the block cannot be reconstructed, and both outcomes are verifiable against the embedded data.

Even a perfect detector answers only half of the practitioner's question, since knowing which pixels of a radiograph were altered does not restore the diagnostic content that was overwritten. Passive restoration is also necessarily generative: an inpainting network fills the region with plausible content that bears no evidentiary relationship to the original. When the artifact is evidence, that is worse than no restoration, because a fabricated region that looks natural invites trust that the detector's flag was meant to withdraw. What is required is a mechanism by which the original content, even as a coarse approximation, survives the attack.

Self-embedding, introduced by Fridrich and Goljan [1], provides exactly this: a compressed representation of each region is hidden elsewhere within the same image, so that a spatially localized attack destroys a region but not its backup. This report describes a reproducible pipeline built on that principle, which evaluates localization and recovery jointly. Two further arguments bear on deployment. Learned detectors

need annotated corpora and accelerator hardware that excludes embedded devices and hospital gateways, and their output is not an argument under cross-examination, since a per-pixel confidence of 0.78 cannot be recomputed or defended line by line.

The second argument concerns reporting practice. The literature habitually reports localization and recovery separately, which lets a healthy in-region reconstruction figure flatter a scheme whose recoverable fraction is undisclosed, since it describes only the blocks that happened to be recoverable. This project therefore treats the joint report of imperceptibility, localization and recovery, with the unrecoverable fraction stated openly, as a requirement of the evaluation not an optional extra.

1.2 LITERATURE SURVEY

1.2.1 Passive and Active Authentication

Passive methods [2], [3] need no cooperation from the acquisition device, an advantage no active scheme can match. The price is annotated training corpora, GPU inference at run time, and restoration that is hallucinated rather than recovered. A learned detector also generalizes only as far as its training manipulations. Active watermarking inverts the robustness objective of Cox et al. [6] into fragility [4], because failure to verify is itself the authentication signal. The survey framing follows [5] and [25].

Pure fragility cannot separate a malicious splice from a benign re-save, since both disturb the embedded bits. The semi-fragile pattern [8] addresses this by tolerating compression-level change while still flagging content change. This project is deliberately fully fragile: it protects images inside a lossless custody chain, and the semi-fragile mode is treated as future scope.

1.2.2 Self-Embedding and Its Known Failure Modes

Fridrich and Goljan [1] are the direct ancestor of this design: block-wise compression, a key-dependent mapping of each block to a distant carrier block, and embedding in the least significant bits. Hierarchical verification [10] improves localization resolution at the cost of a higher payload per block. Security analysis [7], [9], [26] shows that a block watermark depending only on its own content lets an adversary collage a counterfeit that verifies everywhere. The authentication tag adopted

here therefore binds the block content, the block index and an image-level identifier under a secret key.

A second failure mode is the tamper coincidence problem: under a one-to-one backup allocation, a block and the block that holds its backup may be destroyed together. Aminuddin and Ernawan named the problem and addressed it in AuSR3 [15] by mapping each block to the most distant available location. Their earlier AuSR1 [13] established the 2x2 recovery granularity and LSB shifting, AuSR2 [14] gave a texture-preserving variant, and [16] proposed metrics for the coincidence problem. The distant-mapping principle is theirs, not this project's.

A third line of work shares capacity across blocks. Zhang et al. [11], [19] share reference information so that reconstruction becomes the solution of a linear system, and Korus and Dziech [12] add content-adaptive allocation with a formal rate-distortion analysis. Such schemes degrade gracefully and so report far higher whole-image reconstruction at high damage than a one-to-one mapping, which fails all-or-nothing for each block. They pay with a more complex allocation and a global decoding step, whereas the one-to-one mapping adopted here keeps every block independently verifiable and every recovery a single, inspectable lookup. The limitation is real, and reference sharing is the principal upgrade path.

1.2.3 Learned Proactive Watermarking

A recent family embeds a learned watermark so that a trained network both localizes and recovers. EditGuard [17] jointly optimizes an invertible watermarking network for localization and copyright protection, and [18] reports a recent multi-feature fragile scheme. The statement that active watermarking detects but does not recover is therefore no longer accurate. This project's distinction is auditability: a decision here is one recomputed keyed hash, whereas a learned localizer's decision cannot be recomputed without the model weights, and its networks must be trained, versioned and run on suitable hardware. Table 1.1 summarizes the surveyed literature.

Table 1.1: Summary of the surveyed literature

Ref.	Author(s), Year	Technique	Limitation / gap
[1]	Fridrich and Goljan, 1999	Origin of self-embedding; compressed backup at a distant carrier	Recovery approximate; no keyed binding of the block index
[6]	Cox et al., 1997	Spread-spectrum robust	Robustness inverts

		watermark in significant coefficients	authentication; cannot signal change
[7]	Holliman and Memon, 2000	Counterfeiting attack on block-wise independent marks	An attack, not a scheme; forces index and image-ID binding
[8]	Lin and Chang, 2000	Semi-fragile authentication via DCT coefficient-pair invariants	Tolerant to compression, weaker localization precision
[4]	Wong and Memon, 2001	Block-wise fragile signatures in the LSBs; blind and localized	Authenticates only; no content recovery
[9]	Fridrich, 2002	Security analysis of localizing fragile marks	Analysis only; no recovery mechanism
[10]	Lin et al., 2005	Hierarchical multi-level verification	Higher payload cost per block
[11]	Zhang et al., 2011	Reference sharing; reconstruction as a linear system	Higher decode complexity; needs a global solve
[12]	Korus and Dziech, 2013	Content-adaptive self-embedding with rate-distortion analysis	Substantially more complex allocation
[13]	Aminuddin and Ernawan, 2022	AuSR1: 2x2 recovery granularity, LSB shifting	Shifting perturbs bits that an MSB tag must cover
[15]	Aminuddin and Ernawan, 2023	AuSR3: distant block mapping against tamper coincidence	Comparable figures not citable without re-implementation
[17]	Zhang et al., 2024	EditGuard: learned proactive watermark; localizes and recovers	Requires training and weights; not independently recomputable

1.2.4 Market Survey and Research Gap

Deployed products fall into four groups, and none offers localization and recovery from inside the image. Provenance metadata is the first. The C2PA Content Credentials specification [24] attaches a signed manifest recording an asset's origin and edit history, but the manifest lives beside the pixels, so re-encoding or a metadata strip removes it, and it can never restore content. Controlled-capture services such as Truepic authenticate images at capture through a proprietary pipeline, depend on vendor infrastructure, and do not recover destroyed content.

Passive forensic tools such as Amped Authenticate and FotoForensics apply heuristics to arbitrary images, but require expert interpretation, carry no ground truth and offer no restoration. Whole-file digital signatures, as standardized for DICOM medical imaging, detect that a file has changed but neither localize the change nor

recover from it. The market gap therefore mirrors the gap in the literature: existing products either attest to provenance externally or analyse pixels after the fact, and none embeds recoverable content inside the image.

The gap addressed here is deliberately narrow. The distant-mapping idea is credited to AuSR3 [15], and superiority over learned proactive watermarking such as EditGuard [17] is not claimed, since those systems already localize and recover. The contribution is threefold. First, determinism and auditability: any third party holding the key can repeat each detection bit for bit, with no training data, no GPU and no run-to-run variance. Second, an explicit keyed construction in which every bound field is analysed together with the attack it defeats. Third, joint evaluation of imperceptibility, localization and recovery, with the recoverability rate and the unrecoverable fraction disclosed rather than excluded, across four tamper classes and seven tamper ratios against exact ground-truth masks.

1.3 ORGANIZATION OF THE REPORT

Chapter 2 gives the problem definition and scope, including what is deliberately excluded. Chapter 3 presents the project plan and risk analysis, and Chapter 4 the software requirements. Chapter 5 describes the system design, including the mathematical model, keyed construction and UML diagrams. Chapter 6 discusses advantages, limitations and applications, with the limitations given the same prominence as the strengths. Chapter 7 reports the results to date, the conclusions and future work; the reader interested only in the evidence may proceed directly to it. Appendix A contains the feasibility and complexity-class analysis, Appendix B summaries of the referred papers, and Appendix C the plagiarism report.

CHAPTER 2
PROBLEM DEFINITION AND SCOPE

2.1 PROBLEM STATEMENT

Given a digital image stored in a lossless container and a secret key held by the authenticating party, the problem is to embed authentication and recovery information into the image itself such that (a) any subsequent region-level modification is localized to the block grid without reference to the original image or any external database, (b) the original content of the modified region is approximately restored from information that survived within the same image, and (c) any region that cannot be restored is identified as such rather than reconstructed by inference. The statement fixes what the scheme must deliver, not how the payload is arranged.

Three standing constraints apply. Embedding must be perceptually transparent. Verification must be deterministic and independently recomputable, so any party holding the key can repeat a decision bit for bit. The pipeline must run on a commodity CPU with no trained parameters, which excludes any training set, GPU or random initialization.

Clause (a) removes the original image and any registry from the verification path, since a scheme that needs a reference copy has already assumed what it is trying to establish. Clause (b) keeps the repair information inside the same pixels, so the image is self-contained once it leaves the authenticating party. Together they reject schemes that detect without recovering and schemes that recover from a side channel that can itself be lost or altered.

Clause (c) is the least conventional and the most important for the intended use. A scheme that silently inpaints what it could not recover produces an output in which genuine and invented pixels are indistinguishable. Explicit unrecoverable marking turns a gap in the evidence into a documented gap, and that disclosure, rather than visual quality, keeps the output admissible as forensic material.

2.2 GOAL AND OBJECTIVES

The goal of the project is to build and measure a deterministic, training-free fragile watermarking scheme that localizes tampering at block granularity, restores what it can

from within the image, and reports honestly what it cannot. The six objectives below decompose that goal.

- O1: Design a per-block payload that fits entirely within two least-significant bit-planes, so that embedding distortion is analytically bounded, and achieve mean imperceptibility of at least 40 dB PSNR over a standard corpus.
- O2: Construct a key-seeded block mapping that is provably bijective, free of fixed points and short cycles, and enforces a minimum spatial separation between a block and its recovery partner.
- O3: Localize region-level tampering at block granularity with block-level recall of 1.0 across four tamper classes and seven tamper ratios.
- O4: Produce zero false positives on unmodified watermarked images, verified over more than one million independent block verifications.
- O5: Reconstruct flagged regions from partner-held descriptors, report an explicit recoverability rate, and mark unrecoverable regions rather than fabricating content.
- O6: Deliver a reproducible evaluation grid and working software, gated by automated checks that must pass before any result is published.

Each objective carries its verification method: PSNR and SSIM over the 32-image corpus for O1, structural verification of the permutation for O2, the localization grid and the null condition for O3 and O4, the recoverability rate against its structural bound for O5, and the automated sanity gate, which must run before any table or figure is emitted, for O6. An objective with a threshold and a procedure can be closed or visibly left open, which an intention such as improving robustness cannot, so Chapter 7 reports against the objectives one for one, and a result that falls short is reported beside its objective and not deferred to a limitations section.

2.3 STATEMENT OF SCOPE

The project covers still images, in grayscale and three-channel color at 8 bits per channel, stored in lossless containers (PNG, BMP and TIFF). Two block sizes are supported, 8 as the main configuration and 4 for an ablation. Of three recovery descriptor variants, A and C form the main grid and B is an ablation. The evaluation applies four tamper classes, namely copy-paste splicing, object removal by inpainting, crop-and-refill, and noise corruption, each at seven tamper ratios and against exact

ground-truth masks. The corpus contains 32 images, 8 from the USC-SIPI database [22] and 24 from the Kodak suite [23]. A local web application and a demonstration interface wrap the same pipeline.

The tamper ratios are nominal values, and Table 2.1 lists them beside the ratios actually achieved. Five of the seven match their nominal value within 0.005. The two highest are nominal only: the region generator cannot reliably place one compact region over that much of a frame, so a nominal 0.60 yields an achieved ratio of 0.573 and a nominal 0.70 yields 0.653. Results at those settings are reported against their nominal labels with this qualification, so the highest-ratio rows are not read as more demanding than they were.

Table 2.1: Tamper classes and tamper ratios used in the evaluation

Nominal ratio alpha	Achieved ratio (mean ± s.d.)	Pixel-level precision
0.10	0.100 ± 0.000	0.9248
0.20	0.200 ± 0.001	0.9481
0.30	0.300 ± 0.001	0.9581
0.40	0.398 ± 0.005	0.9643
0.50	0.492 ± 0.018	0.9701
0.60	0.573 ± 0.046	0.9740
0.70	0.653 ± 0.064	0.9797

Fig. 2.1 is a qualitative strip of the pipeline end to end: an image is watermarked, tampered, verified so that the affected blocks are localized, and restored, with any block that cannot be rebuilt left visibly marked, as in the quantitative grid.



Fig. 2.1: The four tamper classes applied to a watermarked image, shown here at a 30% tamper ratio.

Several exclusions are equally deliberate. Video and audio are excluded because the block grid, the capacity budget and the ground-truth masks are defined on a single frame. Robustness to lossy re-encoding, resizing, rotation or filtering is excluded

because the scheme is fully fragile by design: a watermark that survived such operations would also survive a malicious edit that mimics them. Adversaries holding the secret key are excluded because no active scheme can defend against them. Real-time camera-firmware embedding is an integration task, and competing with learned detectors on accuracy is excluded because the contribution is determinism and auditability.

2.4 METHODOLOGIES OF PROBLEM SOLVING AND EFFICIENCY ISSUES

The image is partitioned into non-overlapping blocks. The two-LSB capacity of each block is split into a keyed authentication tag and a compressed recovery descriptor of a spatially distant partner block chosen by a key-seeded permutation (detail in Chapter 5). Verification recomputes each tag and compares it with the stored value, localizing tampering to mismatched blocks. Recovery rebuilds a flagged block from the descriptor held by its surviving partner, and a flagged block whose partner is also flagged is marked unrecoverable.

The scheme is single-pass. The tag covers only the six most-significant bit-planes of the block, so writing the payload into the two least-significant planes cannot invalidate what was signed; a scheme whose authentication data depends on the pixels that carry it must iterate until the two agree. Each block costs one keyed hash [20] and one BxB transform, so the total work is O(MN) for an M by N image, with no training. On a commodity AMD x86-64 CPU, single thread and in Python, embedding a 512x512 three-channel image takes 473 ms and verification takes 197 ms, with no GPU. Memory is bounded by the image plus one permutation of K integers, where K is the number of blocks.

The design makes three efficiency trade-offs. The first is block size against localization precision and payload. Moving from B = 8 to B = 4 raises pixel-level precision from 0.9599 to 0.9794, but in-region recovery PSNR falls from 29.63 dB to 26.78 dB, a loss of 2.85 dB. Per-block payload falls from 128 to 32 bits and the tag halves from 32 to 16 bits, so the per-block false-accept probability rises from 2^-32 to 2^-16, an expected 0.25 false accepts per 512x512 image with 16,384 blocks, reported as a finding rather than hidden.

The second is LSB shifting against LSB replacement. A descriptor-carrying pixel is moved to the nearest value whose low bits already match, giving a mean distortion of approximately 0.5 LSB against approximately 1.5 LSB for replacement, at the cost of a more involved embedding routine. Tag-carrying pixels are never shifted, so the verification value is always written exactly as computed: the descriptor tolerates a small perturbation of its carrier pixel, whereas the tag tolerates none.

The third is one-to-one mapping against reference sharing. A one-to-one mapping, in the manner of the distant-mapping idea of [15], is cheap and analyzable, since the recoverability rate has a simple structural bound, but recovery fails all-or-nothing per block. Reference-sharing [11] and fountain-coded [12] schemes degrade more gracefully and reach higher whole-image reconstruction at high tamper ratios. The project accepts that cost knowingly and names reference sharing as its principal upgrade.

CHAPTER 3
PROJECT PLAN

3.1 PROJECT ESTIMATES

The project is a research-and-implementation effort. Its principal deliverable is a working codebase together with a measured evaluation of that codebase, and a deliverable of this kind can be counted in source lines. For that reason the estimate reported here is a basic COCOMO calculation over the code as built, in place of a function-point count made speculatively before the work began. The estimate is therefore retrospective by construction, and it is offered as a consistency check on the effort invested, not as a prediction. Counting only non-comment, non-blank lines of Python, the codebase measures 9,929 lines across 24 modules, which is 9.929 KLOC. Table 3.1 groups these lines by module role.

Table 3.1: Measured size of the implementation, by module group

Module group	Modules	Lines (non-comment, non-blank)
Core scheme (payload, blockmap, embed, detect, recover)	5	1,478
Experiment harness (tamper, metrics, run_experiments, sanity_gate)	4	1,767
Tables, plots and document builders	5	2,251
Support (imageio_any, fit_variant_c, fountain, test_e2e)	4	910
Applications (Flask web application, Streamlit demonstration)	3	2,248
Presentation builders	3	1,275
Total	24	9,929

The model applied is basic COCOMO in organic mode. Organic mode is the correct choice because the team is small, the application domain is well understood, the requirements are stable, and the project carries no hard real-time or rigid interface constraints. With size S in KLOC, effort E in person-months and development time D in months, the constants a = 2.4, b = 1.05, c = 2.5 and d = 0.38 give the following two relations.

$$E = a \times S^{b}$$

$$D = c \times E^{d}$$

Table 3.2: Basic COCOMO estimate, organic mode (a = 2.4, b = 1.05, c = 2.5, d = 0.38)

Quantity	Relation	Value
Measured size	KLOC	9.929
Effort	E = 2.4 x KLOC^1.05	26.73 person-months
Development time	D = 2.5 x E^0.38	8.71 months
Average staffing	E / D	3.07 engineers
Productivity	KLOC x 1000 / E	371 lines per person-month

Substituting the measured size, the effort is E = 2.4 x (9.929)^1.05 = 26.73 person-months, and the development time is D = 2.5 x (26.73)^0.38 = 8.71 months. Dividing effort by time gives an average staffing of 26.73 / 8.71 = 3.07, that is, approximately 3 full-time engineers. Dividing the measured size by the effort gives a productivity of 9,929 / 26.73 = 371 lines per person-month. Table 3.2 collects these figures.

The estimate has to be reconciled with the team that actually exists. The project is executed by a four-student group working part-time alongside coursework across two semesters. An effort of 26.73 person-months over 8.71 months implies about three people working full time, whereas four students each give only a fraction of their week to the project. These facts are consistent with the organic-mode estimate once part-time availability is accounted for. COCOMO was calibrated on professional projects, so it is used here as an order-of-magnitude check and not as a precise measure of student effort.

The remaining cost picture is non-monetary. No licensed software is used, because every dependency is open source. No GPU is required, no cloud compute is rented and no paid dataset is purchased, since the corpus comes from the freely available USC-SIPI [22] and Kodak [23] collections. The direct project cost is therefore effectively the cost of student time and a commodity workstation.

3.2 RISK MANAGEMENT WITH RESPECT TO NP-HARD ANALYSIS

The risk picture depends on the complexity class of the problem, so that question is settled first. Embedding and verification each perform one keyed hash and one BxB transform per block, plus O(K) work to build and apply the block mapping, where K is the number of blocks. The total is O(MN) time for an M x N image, with no iteration,

no optimization and no training. The problem solved here is therefore of class P, and no NP-hard sub-problem arises anywhere in the pipeline. The full argument is given in Appendix A.

One nuance deserves honest statement. Choosing an optimal descriptor allocation, meaning the assignment of backup descriptors to host blocks that maximizes expected recoverability under arbitrary tamper geometry, is a combinatorial assignment problem, and this work does not solve it optimally. The project sidesteps it with a constructive key-seeded permutation that satisfies the separation constraint by construction. The absence of an intractable core removes the single largest schedule risk that a research project can carry, because no part of the work depends on a search whose runtime cannot be bounded.

The remaining risks are ordinary engineering and research risks. Table 3.3 records them as a register giving the category, probability, impact and mitigation of each. The ratings are judgments, not measurements. Three of the risks either materialized or came close, and they are discussed in turn below.

Table 3.3: Risk register

Risk	Prob.	Impact	Mitigation and status
Tamper coincidence: a block and its recovery partner destroyed together	High	High	Minimum-separation mapping; the recoverability rate is reported rather than concealed. Structural, accepted and measured.
MSB-preserved miss: an inpainted block whose six most-significant bit-planes are unchanged	Low	Medium	Predicted by the design analysis, observed as 17 blocks across the main grid and disclosed as a soft gate warning.
Result drift between the prose and the measured grid	Medium	High	Every table and figure is generated from the result file; the gate refuses to emit them on a failure.
Capacity exhaustion at small block sizes	Medium	Medium	Bit budget is analytic; the B = 4 ablation reports the halved tag and its false-accept consequence.
Dependency or environment drift breaking reproduction	Medium	Medium	Versions pinned exactly; the corpus is pinned by SHA-256.

Scope creep into robustness or video	Low	Medium	Scope fixed in Chapter 2; the semi-fragile mode is deferred to future work.
Schedule slip from late measurement	Low	High	The full grid was completed in phase 5, before the report was written.

The first is tamper coincidence. A flagged block can be rebuilt only if its partner survives, so when the tampered region covers both a block and the partner that carries its descriptor, that descriptor is lost. The limit is structural, with expected recoverability near 1 - alpha, and no implementation effort can remove it. The minimum-separation mapping, with d_min = 0.25 x min(M,N), makes it unlikely that one compact region covers both. The recoverability rate rho is also disclosed with every result: it runs from 0.9590 at a nominal ratio of 0.10 to 0.3579 at 0.70.

The second is the MSB-preserved-miss block. The tag binds only the six most-significant bit-planes, so a modification that leaves those planes unchanged escapes detection. The design analysis predicted this residual and measurement then confirmed it: 17 blocks across the main grid were missed, all under inpainting, which lowers recall for that class to 0.99998. The sanity gate reports it as one of its two soft warnings, and the project documents it instead of hiding it.

The third is result drift between the prose of the report and the measured grid, in which a number is copied once, the experiment is rerun and the two then disagree. The mitigation is mechanical. Every table is generated from the result file output/runs.csv, and no table or figure is emitted unless the automated gate runs, with 25 of 27 checks passing.

3.3 PROJECT SCHEDULE

The work is organized as ten phases. Phases 1 to 6 and phase 8 are complete. Phase 7, paper preparation and internal review, is in progress. Phase 9, the reference-sharing extension, and phase 10, the final report, demonstration and viva, are planned. The sequence is not strictly linear, since phase 8 was completed while phase 7 remains open and the two do not depend on one another. Table 3.4 lists each phase with its activity, deliverable and status.

Table 3.4: Plan of work and status

Phase	Activity	Deliverable	Status
1	Literature survey and problem formulation	Survey and gap statement	Complete
2	Scheme design: bit budget, tag construction, block map	Design specification	Complete
3	Implementation of the core pipeline	Embed, detect and recover modules	Complete
4	Tamper harness and metrics	Four tamper classes with exact masks	Complete
5	Full experimental grid and validation	3,232-row grid; 25 of 27 gate checks	Complete
6	Applications and adversarial testing	Web application; two defects found and fixed	Complete
8	Variant C consolidation into the reported grid	Updated tables and figures	Complete
7	Paper preparation and internal review	IEEE-format manuscript	In progress
9	Reference-sharing extension	Prototype and comparison	Planned
10	Final report, demonstration and viva	Report, presentation, demonstration	Planned

Fig. 3.1 presents the same schedule as a Gantt chart, which makes the overlap between phases visible and shows the remaining work concentrated at the end of the plan. The schedule is deliberately front-loaded. The full experimental grid was completed before the report was written, so every claim in Chapter 7 was measured before it was described, not after. This ordering avoids a familiar failure of student projects, in which the text is written first and the experiments are then bent to fit it.

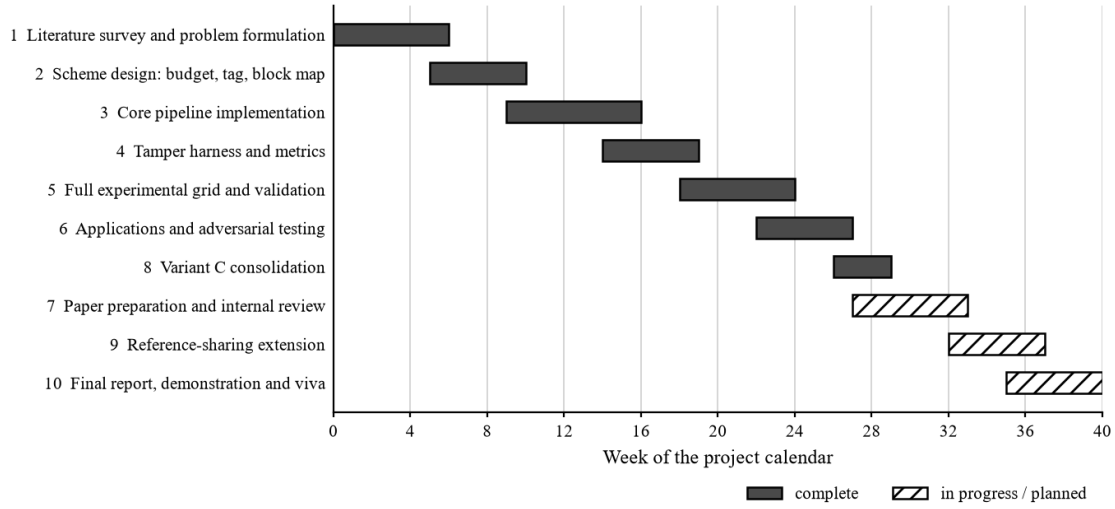


Fig. 3.1: Project schedule. Shaded bars are complete; hatched bars are in progress or planned.

Phase 9 follows directly from the central finding of the evaluation, that mapping coverage and not descriptor fidelity limits recovery quality. It is planned and not yet started, and its absence does not weaken any measured result in this report. Phase 10 depends on the completion of phase 7, because the demonstration and the final report reuse the tables and figures that phase produces from the result file.

CHAPTER 4
SOFTWARE REQUIREMENTS SPECIFICATION

4.1 DATABASE REQUIREMENTS

The core scheme is stateless. Verification needs only the received image, the secret key and the image identifier, with no original image, no reference database and no network call. A third party holding the key can therefore repeat any verdict on an isolated machine. Persistence exists only for the local web application, which keeps a library of protected images so that a user can retrieve, re-download and re-verify what was protected earlier. The library is a single SQLite table named protected, held in the file webapp/library.db. Table 4.1 gives its schema.

Table 4.1: Schema of the protected-image library (SQLite, single table protected)

Column	Type	Purpose
id	INTEGER PK AUTOINCREMENT	Record identifier
name	TEXT NOT NULL	Original file name
created_at	TEXT NOT NULL	Timestamp of protection
height, width	INTEGER NOT NULL	Image geometry
block	INTEGER NOT NULL	Block size used
variant	TEXT NOT NULL	Descriptor variant
key	TEXT NOT NULL	Secret key for this record
key_encrypted	INTEGER NOT NULL DEFAULT 0	Whether the stored key is encrypted
image_id	BLOB NOT NULL	Image identifier bound into every tag
png	BLOB NOT NULL	The watermarked image itself
sha256	TEXT NOT NULL	Digest of the stored PNG
psnr, ssim	REAL	Imperceptibility of this record
blocks	INTEGER	Number of blocks carried
deleted_at	TEXT	Soft-delete marker; the row is retained
Indexes	idx_shape (height, width, block); idx_sha (sha256)	Lookup by geometry and by content digest

Each row records one protected image. The columns name, created_at, height, width, block and variant describe the image and the configuration used to protect it. The image_id column holds the identifier bound into every authentication tag, the png column holds the protected image as a lossless PNG, and sha256 holds its digest. The columns psnr, ssim and blocks record the quality figures measured at protection time

and are nullable. Two indexes are declared, idx_shape on (height, width, block) and idx_sha on sha256, which serve lookups by image geometry and by content digest.

One field is security-relevant. The key column stores the secret key beside the image it protects, and the key_encrypted flag, which defaults to zero, records whether that key is stored encrypted. Storing a key beside the asset it protects is acceptable only for a single-user local demonstration, where the person who protects an image and the person who verifies it are the same and the database file never leaves the machine. In any deployment the key would have to move to an external key store, because anyone able to read the database could otherwise re-embed a forged image. This is a limitation of the demonstration application, not of the scheme.

Retention is soft. The deleted_at column is null for a live record and holds a timestamp once the user deletes it, so a record can still be audited after removal. The same design means the stored image bytes persist after deletion, which would be a privacy consideration for any use beyond the demonstration. Nothing else in the system depends on the database, and losing the file loses the library without affecting the verification of any exported image whose key is known.

4.2 SOFTWARE REQUIREMENTS (PLATFORM CHOICE)

The platform was chosen to keep the trust path short and the evaluation reproducible. Python is the implementation language because the scientific stack lives there and because its standard library supplies hashlib and hmac, so the keyed SHA-256 construction [20] is computed with no third-party cryptographic dependency on the trust path. The standard library also supplies struct, which packs the payload bits. The minimum interpreter version is Python 3.10. Keeping the cryptographic path inside the standard library means an auditor has one fewer package to review before trusting a positive detection, which is the property the scheme exists to provide.

NumPy 2.1.2 provides the vectorized block arithmetic on which the per-block cost depends. OpenCV-headless 5.0.0.93 and scikit-image 0.26.0 supply the transforms and the SSIM implementation [21], and Pillow 10.4.0 handles lossless container formats such as PNG, BMP and TIFF without re-encoding. Matplotlib 3.11.1 draws the figures of the evaluation. Each library does one narrow job, and none of them performs any part of the authentication itself.

Flask serves the local web application because it is small enough to audit in full, which matters for a tool that holds secret keys. Streamlit 1.61.1 provides the demonstration interface over the same pipeline. SQLite stores the library of protected images because the persistence requirement is one table on one machine, and it needs no server process. A heavier database would add an attack surface and an operational burden without adding any capability this project uses.

Every dependency version is pinned exactly, so that the evaluation grid reproduces on another machine, in the same way that every corpus file is pinned by SHA-256. No machine-learning framework appears anywhere in the stack, and the scheme has no trained parameters, which is what allows any verdict to be repeated bit for bit. Table 4.2 lists the software requirements with their roles.

Table 4.2: Software requirements

Category	Requirement
Language	Python 3.10 or later. Cryptographic primitives from the standard library (hashlib, hmac, struct); no third-party cryptography on the trust path.
Core libraries	NumPy 2.1.2, OpenCV-headless 5.0.0.93, scikit-image 0.26.0, Pillow 10.4.0 - pinned exactly.
Interface libraries	Flask (web application); Streamlit 1.61.1 (demonstration interface); Matplotlib 3.11.1 (figures).
Storage	SQLite, for the local protected-image library.
Document build	tectonic and pandoc, optional, only to rebuild the manuscript.
Machine learning	None. No framework, no trained parameters, no model weights.
Operating system	Platform independent; developed and tested on Windows.

4.3 HARDWARE REQUIREMENTS

The scheme needs any commodity x86-64 or ARM CPU and runs on a single thread. No GPU or other accelerator is used at any stage, whether for embedding, verification, recovery or evaluation. Memory is modest and bounded by the image itself plus one permutation of K integers, where K is the number of blocks, which is 4,096 for

a 512x512 image at B = 8. Disk is needed only for the corpus and the result grid, and a few gigabytes is sufficient.

Runtimes were measured on a commodity AMD x86-64 CPU, on a single thread, running the Python implementation on a 512x512x3 image. Embedding takes 473 ms and verification takes 197 ms, so verification costs less than half of embedding. The figures come from an interpreted implementation and are reported as measured, not as a limit on what the scheme can achieve. Table 4.3 summarizes the hardware requirements.

Table 4.3: Hardware requirements

Category	Requirement
Processor	Any commodity x86-64 or ARM CPU, single-threaded.
Accelerator	None. No GPU is required at any stage.
Memory	Bounded by the image plus one permutation of K integers; under 1 GB for the evaluation corpus.
Storage	A few gigabytes for the 32-image corpus, the 3,232-row result grid and the generated figures.
Measured runtime	512 x 512 x 3 image on a commodity AMD x86-64 CPU: embedding 473 ms, verification 197 ms.
Network	None. Verification makes no network call.

Because there is no accelerator requirement, the scheme can run inside an embedded capture device or a hospital gateway, which is exactly where a learned detector cannot. A learned detector needs a trained model and usually a GPU, neither of which fits in a camera body or a clinical network appliance. A deterministic scheme with a few hundred milliseconds of single-thread work per image fits both. The same property keeps the verifier independent of the embedder: a gateway can check incoming images with the key alone, and an auditor can repeat the check on an ordinary laptop without installing any inference runtime.

4.4 ANALYSIS MODELS: SDLC MODEL TO BE APPLIED

The project follows an incremental and iterative model, not a waterfall. The reason is empirical. The payload design, the mapping construction and the descriptor variants were each validated against measurement before the next increment was built, and two

defects found during adversarial testing of the web application fed straight back into the core library. Seven increments were delivered in sequence, each adding one capability to a pipeline that already ran end to end.

- Core payload and tag: key derivation, bit budget, MSB projection, the keyed tag and the descriptor encoding.
- Block mapping: the key-seeded single-cycle permutation with minimum-separation repair.
- Embedding and detection: LSB packing with shifting, and tag recomputation with block localization.
- Recovery: rebuilding flagged blocks from surviving partners and reporting the recoverability rate.
- Tamper harness and metrics: four tamper classes with exact ground-truth masks.
- Full grid and automated gate: the complete experiment and the sanity checks on its output.
- Applications: the Flask web application and the Streamlit demonstration interface.

Every increment ends in a runnable end-to-end test, and no increment is considered complete until the sanity gate runs clean on the results it produces. This rule is what makes the loop iterative in practice. A finding from the harness or the gate returns to the earlier increment that caused it, which is then corrected and re-verified before work moves forward. Fig. 4.1 shows the model as applied here.

Each increment ends in a runnable end-to-end test, and the sanity gate must run clean before the next one begins.

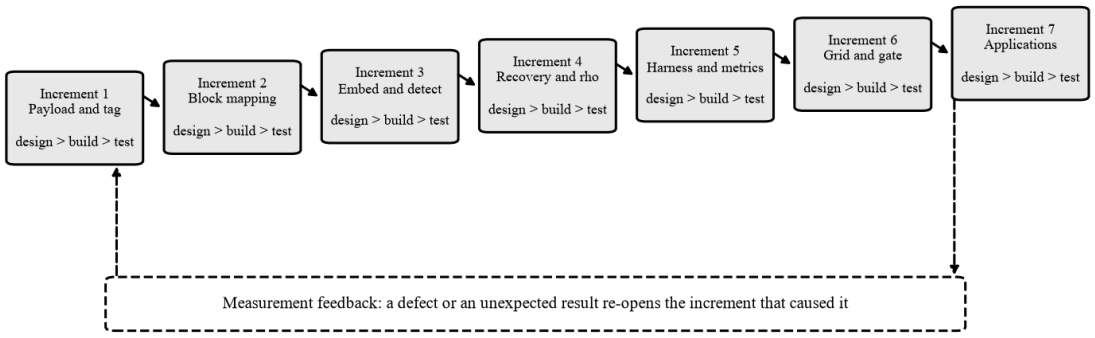


Fig. 4.1: The incremental model as applied: each increment carries its own design, build and test cycle, and measurement feeds back into the increment that caused a defect.

A waterfall sequence from requirements to validation would have served this project badly. It would have discovered the tamper coincidence behavior only at the end, when the mapping design could no longer be changed. Under the incremental model that behavior appeared while the mapping and recovery increments were still open, which is why the minimum-separation repair pass and the disclosure of the recoverability rate are part of the design and not afterthoughts.

CHAPTER 5
SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The system is organized as four decoupled stages. Stage one is embedding, which runs at or near the point of acquisition and turns an original image into a watermarked one. Stage two is distribution, during which the image may be stored, copied, transmitted and, in the adversarial case, tampered with; in this project that stage is simulated by the evaluation harness, which applies four classes of tampering with exact ground-truth masks. Stage three is verification and localization, which can run at any later time and by any party holding the key. Stage four is recovery, which repairs the flagged content where the surviving watermark allows it. The stages communicate only through the image itself and a block mask, so each can be developed, tested and replaced independently. Stages one, three and four depend on the key, while the distribution stage requires no cooperation from the system at all, which is what allows an image to travel through ordinary storage and transfer channels between protection and verification, provided those channels are lossless.

The decisive property of the architecture is what stage three does not need. Verification requires only the received image, the secret key and the image identifier. It needs no original image, no reference database, no network call and no model weights. Every positive detection reduces to one recomputed keyed hash compared against a stored value, so a third party holding the key can repeat the decision bit for bit. This is the practical advantage of an active scheme operating inside an institutional custody chain, and it is the reason the scheme is deterministic and free of run-to-run variance.

Fig. 5.1 shows the data path. The upper row is the embedding stage: the original image I passes through block partition, descriptor encoding, the key-seeded block map and the keyed tag (HMAC) to give the watermarked image Iw. The lower row is the verification and recovery stages: the received image I' passes through LSB extraction, tag recomputation and the tamper mask with refinement, then into the recovery engine, which produces the repaired image. A dashed path from Iw to I' represents distribution. The secret key and the image identifier feed both the block map and the tag stage.

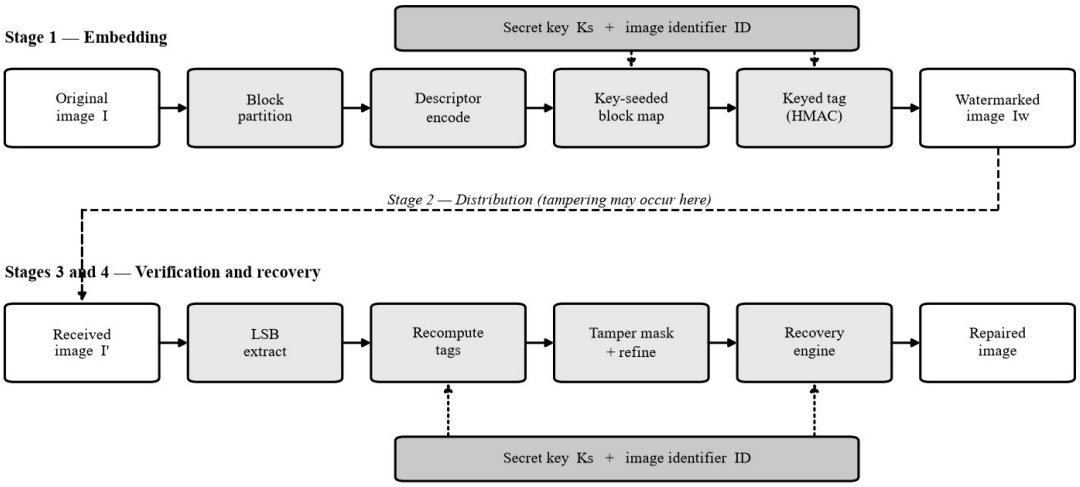


Fig. 5.1: System architecture. The key and the image identifier seed both the block map and the tag; the dashed path carries the distributed image, which may be tampered before verification.

The software mirrors this decomposition. The payload module is the foundation imported by all others: it holds key derivation, the bit budget, block geometry, the MSB projection, tag computation, descriptor encoding and decoding, and LSB packing. The blockmap module builds the key-seeded permutation and verifies its properties. The embed module implements stage one, detect implements stage three, and recover implements stage four. The tamper module generates the evaluation attacks, metrics computes image-quality and localization scores, run_experiments drives the full result grid, and sanity_gate validates that grid before any table or figure is produced. Two applications sit over the same pipeline: a Flask web application with a SQLite library, and a Streamlit demonstration interface. Table 5.2 summarizes the responsibilities.

Table 5.2: Functional modules and their responsibilities

Module	Responsibility
payload	Key derivation, bit budget, block geometry, MSB projection, tag computation, descriptor encode and decode, LSB packing. Imported by every other module.
blockmap	Builds the key-seeded single-cycle permutation, enforces the minimum separation, verifies R1 to R3.
embed	Stage 1. Computes descriptors, permutes them, computes the binding tags, writes both into the two LSB planes.
detect	Stage 3. Extracts the payload, recomputes tags, produces and refines the block-level tamper mask, expands it to a pixel mask, exposes a per-block audit record.
recover	Stage 4. Splits the flagged set into recoverable and

	unrecoverable, rebuilds from partner descriptors, marks the rest, reports rho.
tamper	Generates the four tamper classes with exact ground-truth masks and the achieved ratio.
metrics	PSNR, SSIM, confusion counts, localization scores, in-region recovery metrics.
run_experiments	Executes the full grid and writes the result file.
sanity_gate	Validates the result grid against expected bands and refuses to emit tables or figures on any failure.
Applications	A local Flask web application over SQLite and a Streamlit demonstration interface, both over the same core library.

5.2 MATHEMATICAL MODEL

Let I in {0,...,255}^(MxN) be the host image, Ks the secret key and ID an image identifier. The image is partitioned into non-overlapping BxB blocks in raster order, and b_i denotes the i-th block for i = 0,...,K-1, with c(i) its center coordinate. Dimensions that are not a multiple of B are cropped rather than padded, so no synthetic content is ever authenticated. The model below follows the order in which the quantities are used: first the bit budget that fixes what each block can carry, then the projection that makes embedding safe, then the mapping, the descriptor, the tag, and finally the two measures of behavior, recoverability and cost.

The first quantity is the capacity of a block. Two bit-planes of B^2 pixels give 2B^2 bits, of which a fixed-length tag takes at most half and the descriptor takes the rest.

$$cap(B) = 2B^2, \quad t(B) = \min(32, cap(B)/2), \quad d(B) = cap(B) - t(B)$$

At B = 8 the triple (capacity, tag, descriptor) is (128, 32, 96) bits. At B = 4 the capacity falls to 32 bits and the tag shrinks accordingly, which is the cost of finer localization examined in the ablation. Table 5.1 gives the per-block budget, and Fig. 5.2 shows how the 128-bit payload is divided into a 32-bit authentication tag and a 96-bit recovery descriptor. The descriptor in turn consists of 12 quantized DCT coefficients of 8 bits each, and the whole payload is written into the two least-significant bit-planes of the block's 64 pixels.

Table 5.1: Per-block bit budget

Block size B	cap(B) = 2B^2	Tag t(B)	Descriptor d(B)	Tag false-accept probability
8 (main)	128 bits	32 bits	96 bits	2^-32

4 (ablation)	32 bits	16 bits	16 bits	2^-16

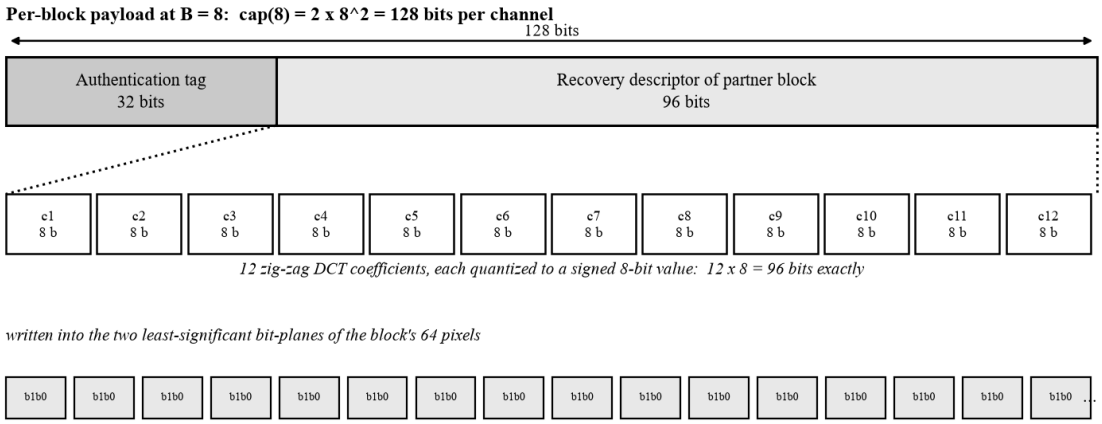


Fig. 5.2: Per-block payload layout at B = 8.

Embedding is made safe by a most-significant projection that separates what is signed from what is overwritten.

$$MSB(a) = a \text{ AND } 0xFC$$

The projection zeroes the two least-significant bits of a pixel value and is idempotent, so applying it twice changes nothing. Embedding writes only to those two bit-planes, so it cannot alter the content that the tag was computed over. That is exactly what makes the scheme single-pass: no embed-and-verify loop is needed, and the tag computed before embedding is the tag that verification will recompute afterwards.

The block map decides which block carries which descriptor. It must satisfy three requirements.

(R1) m bijective; (R2) ||c(i) - c(m(i))||_2 >= d_min; (R3) m is a single K-cycle

The separation is d_min = (1/4) min(M, N). Requirement R3 is obtained by construction rather than by rejection sampling. A cryptographically seeded Fisher-Yates shuffle produces a cycle order of the K blocks, and m is read off that order directly, sending each block to its successor in the cycle. Fixed points and mutual pairs are therefore impossible, and bijectivity holds without a check. A repair pass then enforces R2 by exchanging offending positions, after which all three properties are verified. Fig. 5.3 illustrates the result.

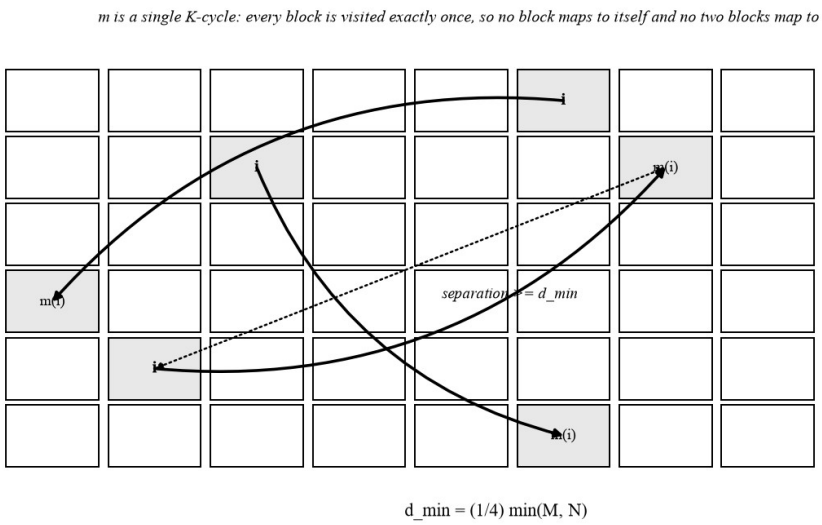


Fig. 5.3: The key-seeded block mapping i -> m(i).

The descriptor is a low-frequency summary of the block's projected content. For Variant A it is the quantized leading portion of the transform.

$$q_hat(i,l) = clip(round(C_i(sigma(l)) / Delta_l), -128, 127), \quad l = 1,...,12$$

Here C_i is the orthonormal two-dimensional DCT of the level-shifted projected block, sigma(l) is the zig-zag index and Delta_l the step size of band l. No retained coefficient can clip. By the Cauchy-Schwarz inequality the magnitude of any coefficient is at most 1024 at B = 8, and every step size is large enough that the quantized magnitude stays inside the signed 8-bit range. Variant C uses the same 96-bit layout with an adaptive, rate-distortion-fitted allocation.

Authentication binds the block's content, its position and the descriptor it carries.

$$a_i = Trunc_32(HMAC\text{-}SHA256_Ks(ID \,||\, i \,||\, MSB(b_i) \,||\, r_\{m\text{^}-1(i)\}))$$

The primitive is HMAC over SHA-256 [20], truncated to 32 bits. Each bound field answers a specific attack. MSB(b_i) makes the tag independent of the bits embedding will write, so the tag survives its own insertion. The block index i and the identifier ID defeat the collage attack of [7], in which blocks from different positions or different images are recombined because each block is authenticated in isolation. The last field, r_{m^-1(i)}, is the descriptor this block physically carries on behalf of another block; binding it prevents an attacker from replacing recovery data without detection, since any substituted descriptor changes the recomputed tag.

Recovery quality is summarized by the recoverability rate. With T the flagged set and U the subset of T whose partner is itself flagged,

$$rho = 1 - |U| / |T|, \quad U = \{ i \in T : m(i) \in T \}$$

A flagged block is unrecoverable exactly when the block that carries its descriptor is also flagged. If the position of m(i) were independent of the tampered region, the probability of that event would equal the tampered fraction alpha, giving the structural bound E[rho] approximately 1 - alpha. The bound is a baseline of the one-to-one allocation, not a guarantee, and the evaluation chapter compares it with measured values.

Finally, the cost of the scheme is linear in the image size.

$$T(M,N) = O(MN)$$

Embedding and verification each need one keyed hash and one BxB transform per block, plus O(K) work to build the mapping. There is no iteration, no optimization and no training, and no NP-hard sub-problem arises anywhere in the pipeline, so the problem is of class P. Memory use is likewise proportional to the image, because blocks are processed independently once the map is built, and the only quantity that must be held for the whole image is the permutation itself.

5.3 DATA FLOW DIAGRAMS

The data flow diagrams decompose the system from its external boundary down to the operations of the detector and the recovery engine. Each level refines exactly one level above it, and the same names are used throughout, so a flow that appears at the boundary can be traced to the operation that consumes it. Throughout, the secret key is treated as an input supplied by the user at the time of each operation and not as data owned by the pipeline.

5.3.1 Level 0 (Context Diagram)

Fig. 5.4 treats the whole system as one process, the Watermark-Guided Tamper Localization and Recovery System. Two external entities interact with it. The Content Owner supplies an original image and a secret key and receives the watermarked image. The Verifier supplies a received image and a secret key and receives a tamper map, a repaired image and an audit record. A single data store, D1 Protected Image Library, holds the protected images.

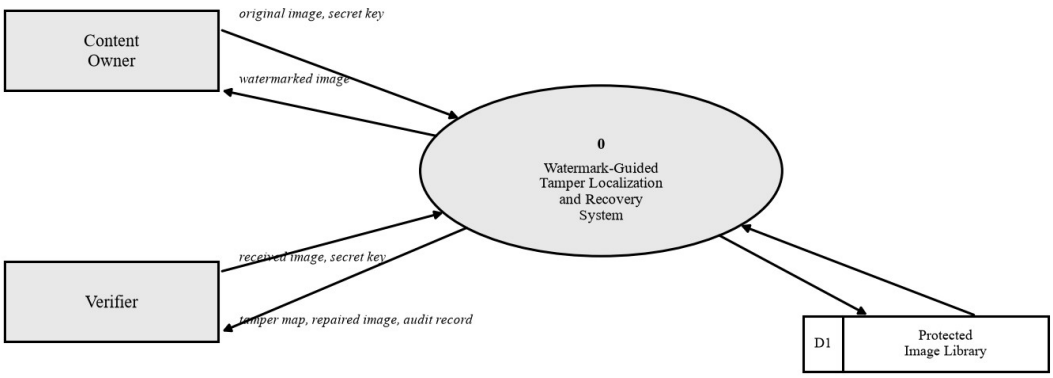


Fig. 5.4: Data flow diagram, Level 0 (context diagram).

5.3.2 Level 1

Fig. 5.5 opens the single process into four. Process 1.0 Embed Watermark produces the protected image; 2.0 Store and Distribute places it in D1 and releases copies; 3.0 Verify and Localize examines a received image; and 4.0 Recover Content repairs it. The key and the image identifier enter processes 1.0, 3.0 and 4.0. The block mask passes from 3.0 to 4.0, and the audit record leaves 3.0 toward the Verifier.

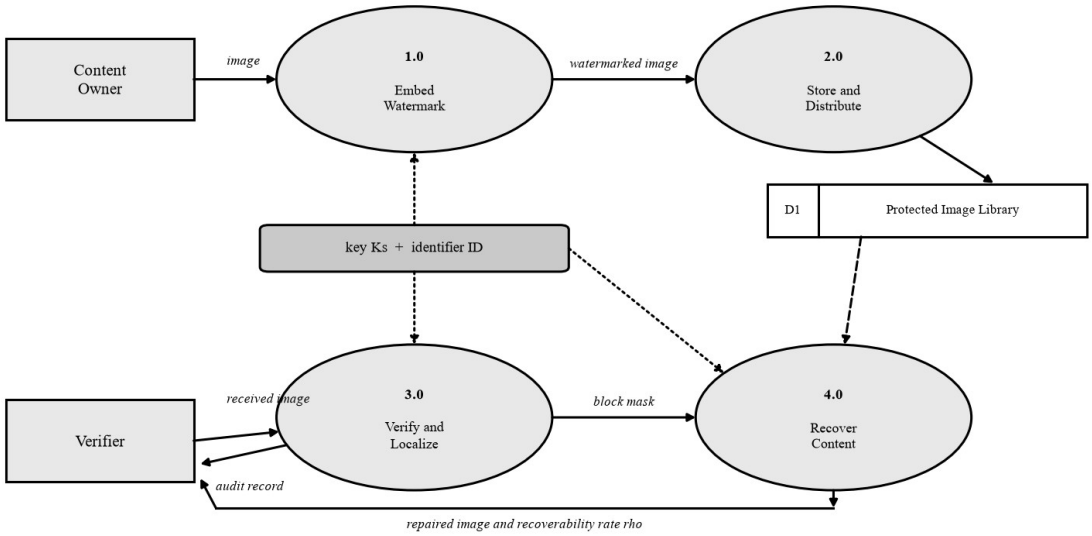


Fig. 5.5: Data flow diagram, Level 1.

5.3.3 Level 2

Fig. 5.6 decomposes processes 3.0 and 4.0. Process 3.0 contains 3.1 Extract LSB Payload, 3.2 Recompute Tags, 3.3 Compare and Build Block Mask, and 3.4 Refine and Expand to Pixel Mask. Tags are compared per channel and combined by an OR rule: a

tamper touching any single channel flags the block. The refinement pass then examines each unflagged block whose neighbors are mostly flagged. Its threshold scales with the number of valid neighbors, so border and corner blocks, which have fewer neighbors, are not penalized.

Process 4.0 contains 4.1 Partition Flagged Set, 4.2 Decode Partner Descriptor, 4.3 Write Recovered Pixels and 4.4 Mark Unrecoverable Blocks. The flagged set is first split into blocks whose partner is intact and blocks whose partner is also flagged. The first group is rebuilt from the partner descriptor; the second is marked explicitly, never inpainted and never left in place silently, so a viewer can always distinguish reconstructed content from content that could not be recovered.

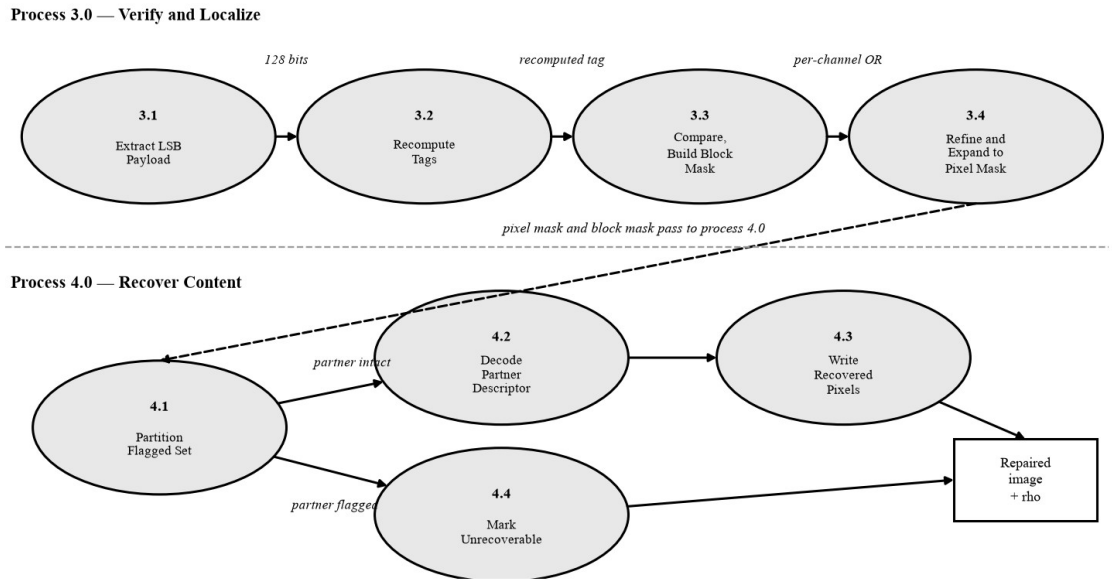


Fig. 5.6: Data flow diagram, Level 2: decomposition of processes 3.0 and 4.0.

5.4 ENTITY RELATIONSHIP DIAGRAM

Fig. 5.7 shows the data model. The central entity is PROTECTED_IMAGE, whose primary key is id. Its attributes record the image name, creation time, dimensions, block size, descriptor variant, the key, the image identifier, the protected image bytes, a content hash, the imperceptibility scores, the number of blocks, a soft-deletion timestamp and a flag stating whether the key is stored encrypted. In the implementation this entity is a single SQLite table with indexes on the image shape and on the hash.

Two further entities appear in the diagram. VERIFICATION_EVENT is related to PROTECTED_IMAGE one-to-many, since one protected image may be verified many times. KEY_PROFILE is related many-to-one, since many images may share one key.

The diagram marks both as conceptual. The cardinalities follow from how a protected image is used: it is embedded once under one key, but it may be checked repeatedly, by different parties and at different times, and each check would produce an event worth recording in an audited setting.

The distinction needs to be stated plainly. Only PROTECTED_IMAGE is persisted in the current build. The two conceptual entities are drawn because a deployment into an archive or an ingest pipeline would need a record of each verification and a managed store of keys, not because the software provides them. In the present software, verification results are returned to the user and are not stored, and keys are held as an attribute of the image record.

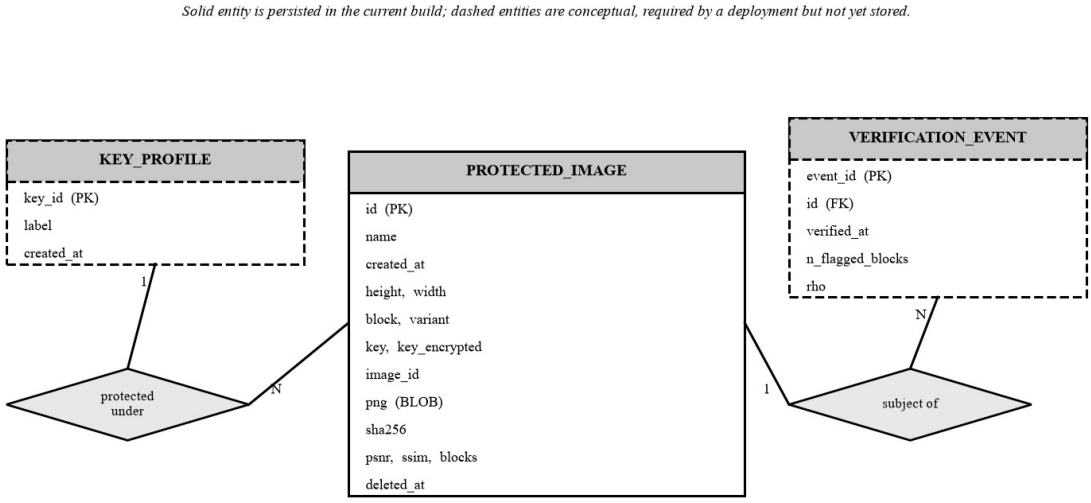


Fig. 5.7: Entity-relationship diagram of the protected-image library.

5.5 UML DIAGRAMS

5.5.1 Use Case Diagram

Fig. 5.8 shows three actors. The Content Owner protects images, individually or in batches, browses the library and downloads protected images. The Verifier applies simulated damage, verifies an image, views the per-block audit, repairs the image and exports a report. The Administrator runs the attack demonstration and deletes records. Verify Image includes Extract Payload, and Repair Image extends Verify Image, since repair is only offered after verification has flagged blocks.

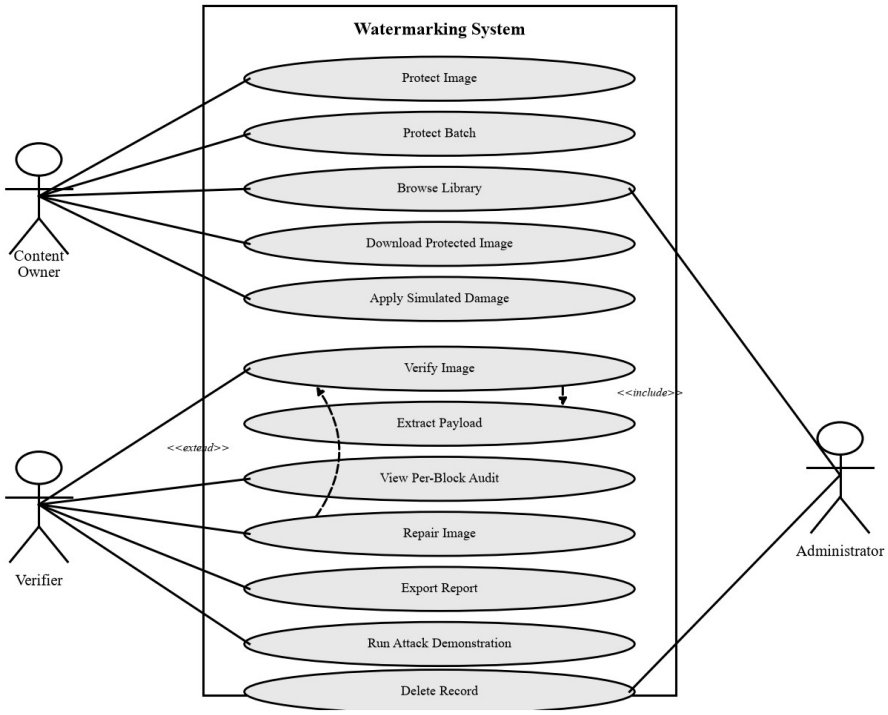


Fig. 5.8: Use case diagram.

5.5.2 Class Diagram

Fig. 5.9 presents the principal classes with their operations and dependencies. PayloadCodec and BlockMap form the foundation. Embedder depends on both to write the payload, and Detector depends on them to read it back, returning a DetectResult. RecoveryEngine consumes that result. TamperHarness, Metrics and ImageIO support evaluation and file handling, and LibraryStore persists records for the web application.

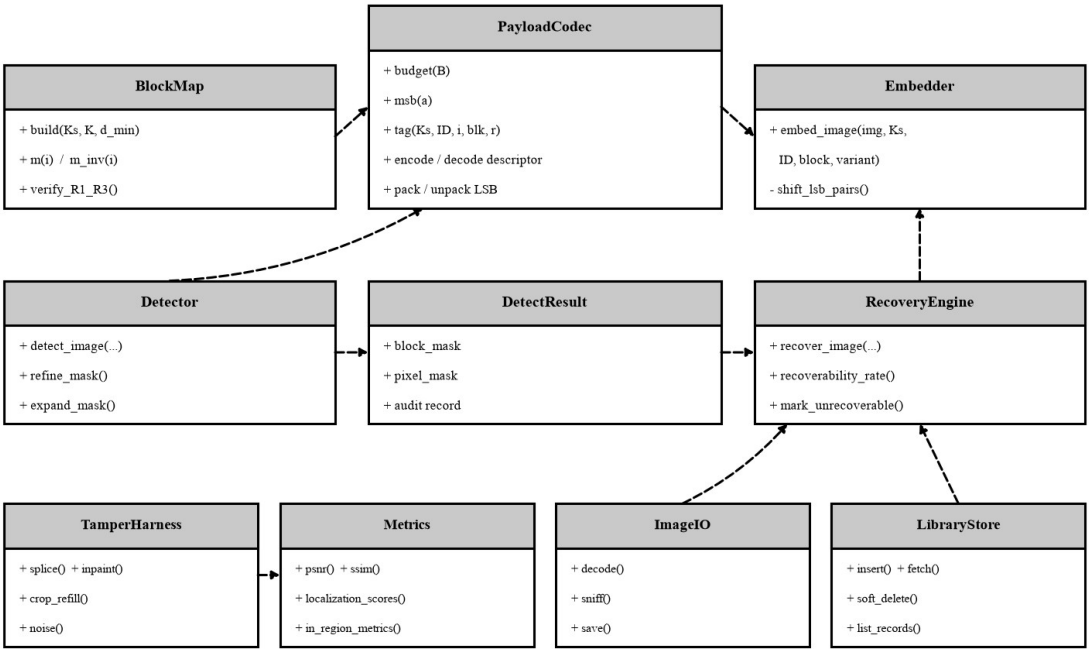


Fig. 5.9: Class diagram. Dashed arrows are dependencies and point at the class whose operations are used.

5.5.3 Sequence Diagram

Fig. 5.10 follows one round trip through the web application. The Content Owner submits an image to the Web Server, which calls the Embedder; the Embedder uses PayloadCodec and BlockMap and returns the protected image, which LibraryStore saves. Simulated damage is then applied, and the Detector verifies the damaged image against the key and identifier. The RecoveryEngine repairs the flagged blocks, and the recoverability rate is returned to the user together with the repaired image.

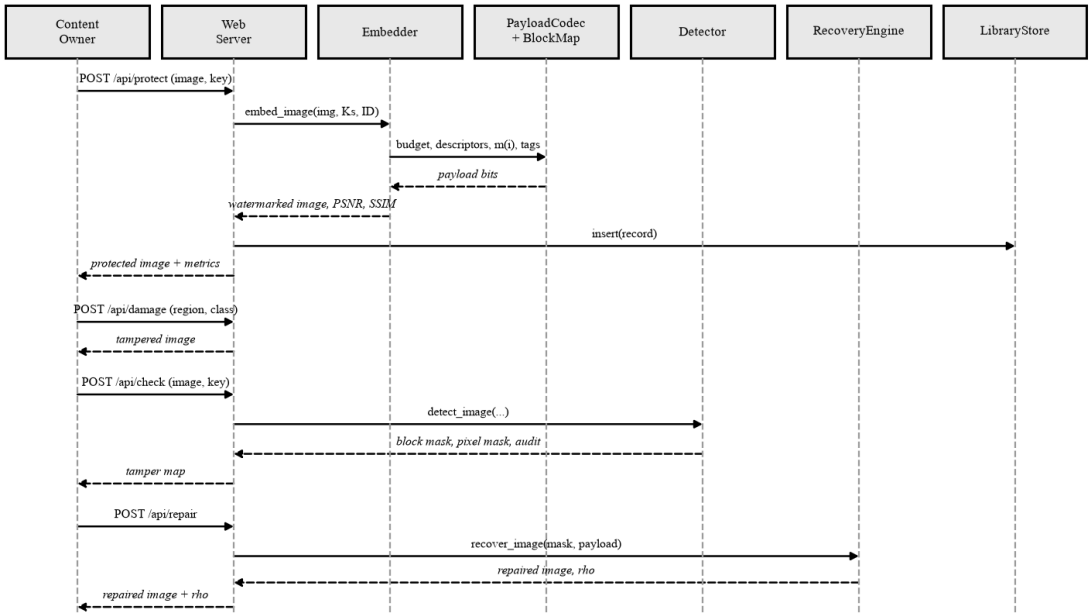


Fig. 5.10: Sequence diagram for the protect, damage, verify and repair round trip.

5.5.4 Activity Diagram

Fig. 5.11 shows verification and recovery as an activity flow. For each block the tag is recomputed and a decision node tests whether it matches the stored tag. Matching blocks are accepted as authentic. Mismatching blocks enter the flagged set, and a second decision node tests, for each flagged block, whether its partner is intact. If so, the block is reconstructed from the descriptor; if not, it is marked unrecoverable. The two paths rejoin before the output is assembled.

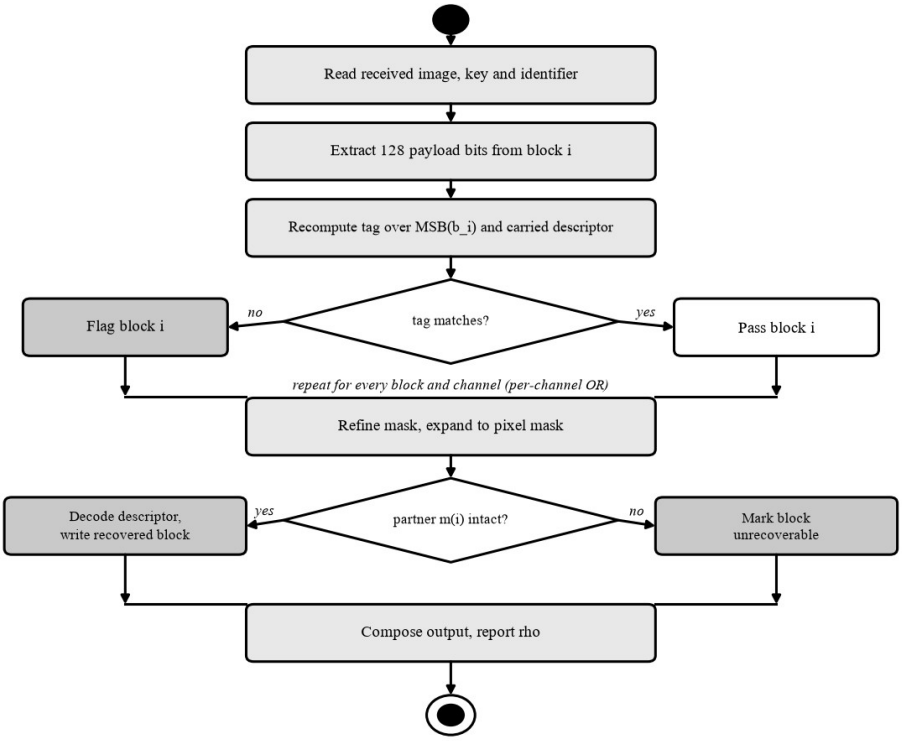


Fig. 5.11: Activity diagram for verification and recovery.

5.5.5 State Machine Diagram

Fig. 5.12 describes the lifecycle of an image through eight states: Unprotected, Protected, Distributed, Under Verification, Authentic, Tampered, Recovered and Archived. Embedding moves an image from Unprotected to Protected, and release moves it to Distributed. Verification leads either to Authentic, when no block is flagged, or to Tampered. Repair moves a Tampered image to Recovered, and either outcome may end in Archived.

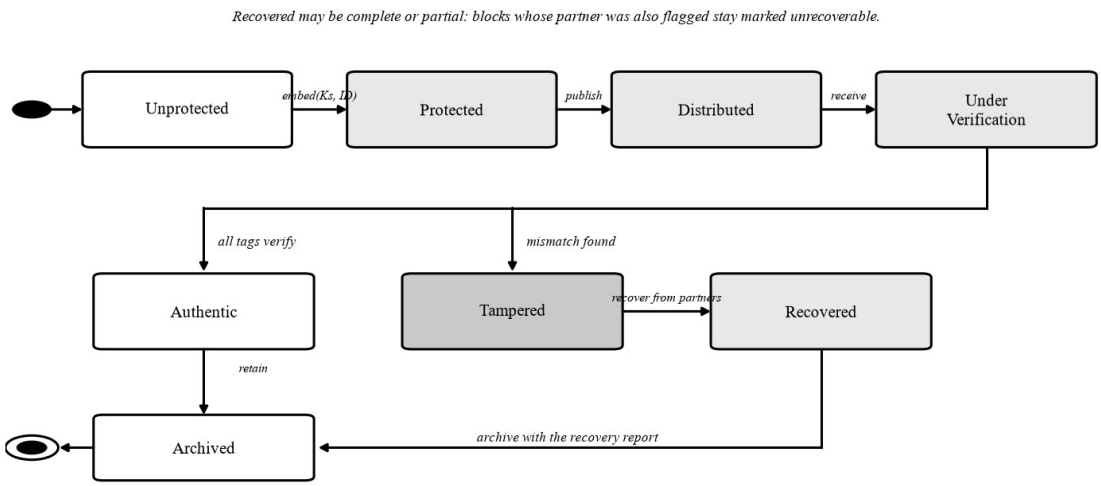


Fig. 5.12: State machine diagram: the lifecycle of a protected image.

5.5.6 Component Diagram

Fig. 5.13 groups the software into components with provided and required interfaces. The Core Library provides the embedding, detection and recovery interfaces. The Experiment Harness and the Flask Web Application, with its browser user interface, require them, as does the Streamlit Demonstration Interface. The web application also requires the SQLite Library for persistence. The Report and Figure Builders consume the harness output. Because every consumer uses the same core interface, no application has its own copy of the algorithm.

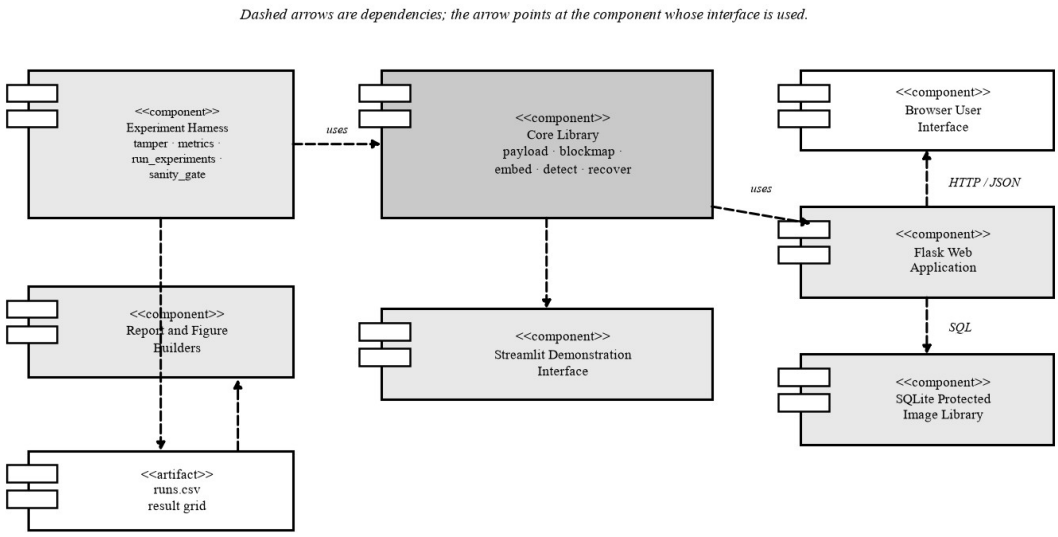


Fig. 5.13: Component diagram.

5.5.7 Deployment Diagram

Fig. 5.14 shows a deployment on a single commodity workstation. The workstation hosts the Python runtime with the core library and the Flask server, the local SQLite file and the browser client. A separate offline node, the experiment harness, runs the evaluation and produces the result grid. No server, no accelerator and no network service is required, which matches the scope of a locally controlled custody chain and keeps the system reproducible.

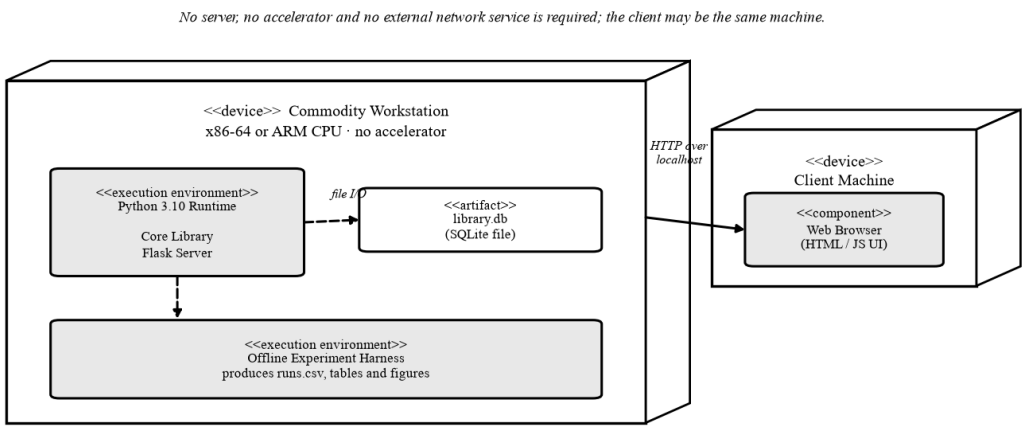


Fig. 5.14: Deployment diagram.

CHAPTER 6
OTHER SPECIFICATION

6.1 ADVANTAGES

The advantage that matters most is auditability. Every positive detection reduces to a single recomputed keyed hash comparison: the verifier extracts the stored 32-bit tag from a block's two least-significant bit-planes, recomputes HMAC-SHA256 [20] over its six most-significant bit-planes, the image identifier, the block index and the partner block's descriptor, and compares the two. Any third party holding the key can repeat that bit for bit, with no training data, accelerator or run-to-run variance, which an evidentiary application requires and a learned detector, whose decision is a score from a trained network [17], cannot offer.

Determinism extends to the whole evaluation. Embedding, verification and recovery contain no iteration, optimization or random initialization beyond the key-seeded permutation, so the 3,232-row grid reproduces exactly on the same pinned corpus. At one keyed hash and one block transform per block, the scheme is in problem class P with O(MN) cost. A disagreement between two verifiers can therefore only come from a different key, a different image or a defect, never from chance. The null-condition result is likewise reproducible across five independent keys, two descriptor variants and thirty-two images, and the corpus is pinned by SHA-256 so an examiner can rebuild it exactly.

Secondary advantages are listed below.

- Self-containment: verification needs only the received image, the key and the image identifier.
- No accelerator: 473 ms to embed and 197 ms to verify a 512x512 colour image on a commodity single-thread CPU.
- Measured imperceptibility: mean PSNR of 45.479 dB for Variant A and 45.492 dB for Variant C, with a minimum SSIM of 0.97831 over the whole B = 8 set.
- Exact block-granularity localization: block-level recall of 1.0000 for three tamper classes and 0.99998 for inpainting, with no false-positive block in 1,802,240 null-condition verifications.

- Honest recovery: regions whose partner block was also destroyed are marked unrecoverable, not filled with fabricated content.
- Analytic predictability: the recoverability rate has a closed-form structural bound, 1 - alpha, against which every measurement can be checked.

6.2 LIMITATIONS

Each limitation is classed as structural, following from this scheme's design choices, or as an engineering gap, closable within the same design, and names where Chapter 7 proposes a remedy.

- Full fragility is structural: any lossy re-encode, resize or filter destroys the watermark, so only images inside a lossless custody chain are protected. Section 7.3 proposes a semi-fragile mode tolerant of benign JPEG re-encoding.
- One-to-one backup allocation is structural and fails all-or-nothing per block, whereas reference-sharing [11] and fountain-coded [12] schemes degrade gracefully and reach far higher whole-image reconstruction at high tamper ratios. Section 7.3 names reference sharing as the highest-value upgrade.
- Recovery is a structural low-frequency approximation from 12 DCT coefficients, not the original block. Section 7.3 addresses this through the same allocation change, which raises coverage.
- Pixel-level precision is capped by block quantization at 0.9599 for B = 8, a structural limit that cannot improve without a finer grid. Section 7.3 proposes hierarchical grids.
- The two highest tamper ratios are an engineering gap: the region generator cannot reliably place one compact region over 60 to 70 percent of a frame, so they are nominal only, at achieved values of 0.573 and 0.653. No scheme change is needed, and Chapter 7 proposes none.
- At B = 4 the tag halves to 16 bits, a structural trade-off raising the per-block false-accept probability from 2^-32 to 2^-16, about 0.25 expected false accepts per 512x512 image. Section 7.3 proposes finer grids avoiding the full tag cost.
- Security rests entirely on key secrecy, which is structural: an adversary holding the key can re-embed a forged image. Section 7.3 proposes a formal security proof of the keyed construction.

- Capacity is fixed and structural: the two LSB planes of a 512x512 colour image hold 1,572,864 bits, or 192 KB. Section 7.3 proposes hierarchical grids to spend it more efficiently.

6.3 APPLICATIONS

The scheme fits applications in which an image is captured once, stored losslessly and may later be challenged. A forensic evidence archive is the clearest case: the keyed tag makes every stored image a self-verifying record, and the per-block audit trail shows which blocks failed. Media authentication at ingest uses the same property, with an organization marking an image as it enters its asset system and verifying it at each reuse. Both depend on determinism, since a result must be repeatable by a party who did not run the original check.

Medical imaging integrity in a picture archiving and communication system (PACS) fits because lossless containers are already the norm for diagnostic images and localization says which region of a scan was altered. Legal and official document scans fit because a scan is produced once under controlled conditions, and a tamper that changes a figure or a signature is localized and, where the partner block survives, partly reconstructed. Insurance claim verification benefits from needing no accelerator, since a claims office can verify photographs on ordinary hardware.

Self-authenticating camera or scanner firmware is the most direct deployment, because embedding at capture removes any gap between acquisition and protection and the firmware carries no trained parameters to maintain or certify. All six domains share the precondition that makes an active scheme viable: an acquisition pipeline under institutional control. An image found on the open internet cannot be protected retroactively, because the watermark must be present before the tampering occurs.

CHAPTER 7
RESULTS, CONCLUSIONS AND FUTURE WORK

7.1 RESULTS OBTAINED TO DATE

The evaluation grid holds 3,232 rows in output/runs.csv. The main grid contributes 1,792 rows: 32 images, four tamper classes, seven nominal tamper ratios from 0.10 to 0.70, and two descriptor variants, A and C, at B = 8. The null condition contributes 320 rows (32 images, two variants, five keys), and the remaining 1,120 rows are ablations covering B = 4 and Variant B. The 32 colour images, 8 from USC-SIPI [22] and 24 from the Kodak suite [23], are pinned by SHA-256. An automated gate passes 25 of 27 checks; the two soft warnings are the inpainting MSB-preserved-miss category and three rows with slightly elevated coincidentally unchanged pixels. Every table and figure is generated from the result file after the gate has run, so the report cannot drift from the measurement.

7.1.1 Imperceptibility

Variant A reaches a mean PSNR of 45.479 dB and a mean SSIM [21] of 0.98847 over the 32-image corpus, and Variant C reaches 45.492 dB and 0.98841. The minimum SSIM over the whole B = 8 set is 0.97831, so no image is visibly degraded. The weakest named image is Tiffany at 44.07 dB for Variant A, against 45.64 to 45.73 dB for the other named images. Table 7.1 summarizes these figures.

Table 7.1: Imperceptibility of watermarked images (32-image corpus, B = 8, LSB shifting)

Descriptor variant	Mean PSNR (dB)	Mean SSIM	Minimum SSIM
Variant A (DCT, 12 coefficients)	45.479	0.98847	0.97831
Variant C (adaptive DCT, main default)	45.492	0.98841	0.97831
AuSR1 [13], as published	45.57	approx. 0.99	-

AuSR1 publishes a watermarked-image PSNR of 45.57 dB [13]; the measured 45.479 dB for Variant A is within 0.1 dB of it. LSB shifting, which moves a descriptor-carrying pixel to the nearest value whose low bits already match, closes most of the gap to plain replacement, cutting mean distortion from about 1.5 LSB to about 0.5 LSB.

7.1.2 Localization

Block-level recall is 1.0000 for copy-paste splicing, crop-and-refill and noise corruption, and 0.99998 for object removal by inpainting. Across the 1,792 main-grid trials, 17 blocks were missed because the inpainting preserved their most-significant bit-planes, the predicted residual of the descriptor-binding design and not an unexplained failure. Pixel-level precision is 0.9599 as a main-grid mean and rises with tamper ratio: 0.9248, 0.9481, 0.9581, 0.9643, 0.9701, 0.9740 and 0.9797 at ratios 0.10 through 0.70. The shortfall from 1.0 is the 8x8 block-quantization penalty, which shrinks as regions grow. Neighborhood refinement flagged only 2 blocks out of 10,092,544 verified.

Block-level precision is deliberately not headlined: a block counts as tampered if the region covers any of its pixels, so a block-level false positive is structurally impossible and the metric reads 1.000 regardless of detector quality. The pixel-level figure is the honest measure of how tightly the flagged area follows the true region.

The null condition tests unmodified watermarked images. Across 320 rows, 1,802,240 independent block verifications produced 0 false-positive blocks, a false-positive rate of 0.000000 with a rule-of-three 95% upper bound of 1.66e-6. This is a determinism and implementation-consistency check across images, variants and keys, not a measurement of the 2^-32 cryptographic false-accept probability, which no experiment of this size could resolve. Table 7.3 gives these figures.

Table 7.3: Null condition: unmodified watermarked images

Quantity	Value
Null-condition rows	320
Images x variants x keys	32 x 2 x 5
Independent block verifications (n)	1,802,240
False-positive blocks (sum)	0
Block-level false-positive rate	0.000000
Pixel-level false-positive rate	0.000000
Rule-of-three 95% upper bound, 3 / n	0.00000166

7.1.3 Recovery

The recoverability rate rho is 0.9590, 0.8584, 0.7479, 0.6371, 0.5335, 0.4437 and 0.3579 at nominal ratios 0.10 to 0.70, against structural bounds of 0.90, 0.80, 0.70, 0.60, 0.50, 0.40 and 0.30. It is monotone, near-linear and 3 to 6 percentage points above its

own bound at every ratio, not collapsing toward it. The main-grid mean is 0.6482. Table 7.2 lists the values, and Fig. 7.1 plots the measured curve with error bars against the dashed 1 - alpha bound and the shaded margin.

Table 7.2: Recoverability rate and in-region recovery quality by tamper ratio (main grid, mean over four tamper classes)

alpha	Achieved	rho measured	Bound 1 - alpha	Margin	In-region PSNR (dB)
0.10	0.100	0.959	0.90	+0.059	30.15
0.20	0.200	0.858	0.80	+0.058	29.70
0.30	0.300	0.748	0.70	+0.048	29.79
0.40	0.398	0.637	0.60	+0.037	29.74
0.50	0.492	0.533	0.50	+0.033	29.67
0.60	0.573	0.444	0.40	+0.044	29.84
0.70	0.653	0.358	0.30	+0.058	29.78

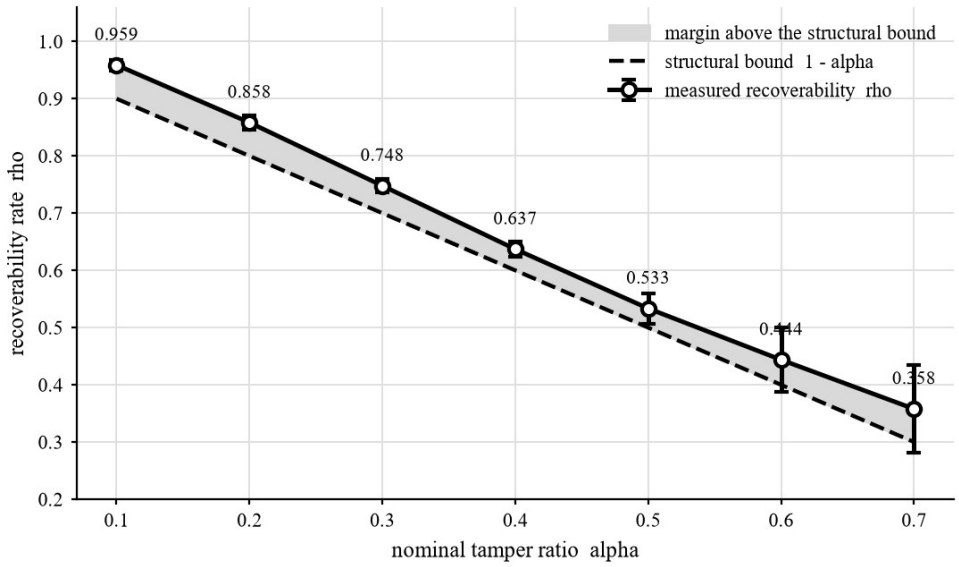


Fig. 7.1: Recoverability rate against tamper ratio. The measured curve stays above its own structural bound at every ratio.

By contrast, in-region PSNR stays essentially flat across ratios, at 28.49 dB for Variant A and 31.13 dB for Variant C, with a main-grid mean of 29.81 dB, while whole-image PSNR under the unmarked convention falls with coverage to a Variant A mean of 21.40 dB. A recovered block is about as good at high tamper ratios as at low ones; what changes is how many can be recovered, so mapping coverage, not descriptor fidelity, is the dominant constraint, pointing directly at reference sharing as the highest-value upgrade. Fig. 7.2 plots in-region PSNR for Variants A and C and whole-image PSNR against tamper ratio.

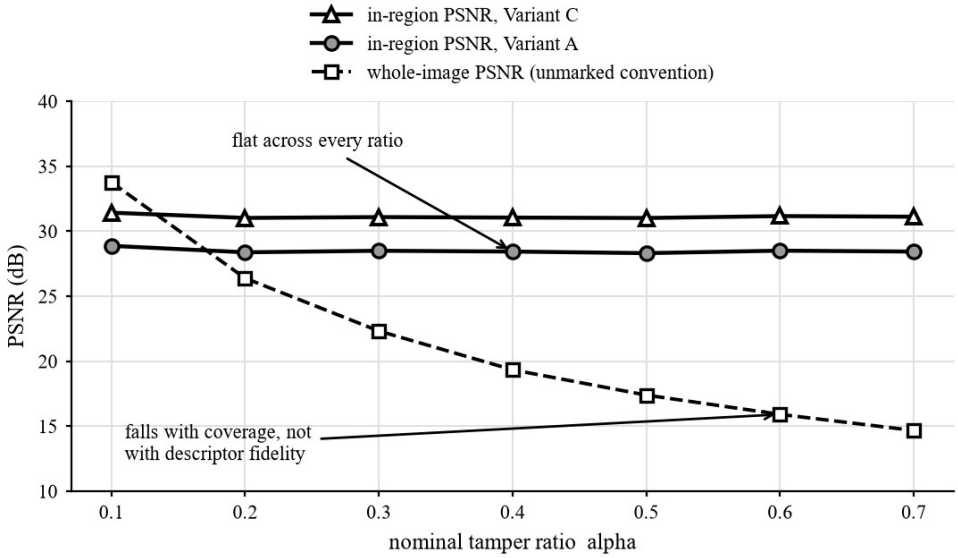


Fig. 7.2: In-region and whole-image PSNR against tamper ratio.

Fig. 7.3 is a qualitative strip from original to recovered image. The watermarked image is indistinguishable from the original, the mask follows the tampered region within a block-wide margin, and recovered content is a smooth low-frequency approximation, with fine texture lost and unrecoverable blocks marked, not invented.



Fig. 7.3: Qualitative result on Lena under copy-paste splicing. The speckles in the recovered panel are blocks marked unrecoverable, not noise.

7.1.4 Comparison and Ablation

Table 7.4 compares the scheme with published work, with figures cited and not re-implemented, so the comparison is indicative only. The entry for this project uses the whole-image unmarked convention, 17.39 dB at 50% tamper for Variant A, the only quantity comparable to the published numbers; the default output, which marks unrecoverable regions rather than fabricating content, gives 12.40 dB at 50% tamper, about 5 dB lower. Fountain-coded reconstruction [12] at 37 dB and the multi-feature scheme of Wu et al. [18] at above 30 dB are well ahead of this scheme on recovery; the contribution here is auditability, not reconstruction.

Table 7.4: Comparison with published self-embedding schemes

Method	WM PSNR (dB)	WM SSIM	Recovered PSNR	Notes

			(dB)	
AuSR1 [13]	45.57	approx. 0.99	27.64	worst case reported; 2x2 blocks
Wu et al. [18]	> 41	-	> 30 at 50% tamper	0% FPR and FNR reported
Korus and Dziech [12]	-	-	37 at 50% damage	fountain codes; over 10,000 images
This work (Variant A, measured)	45.48	0.9885	17.39 at 50% tamper	block precision 1.0000, block recall 1.0000, pixel precision 0.9599; the honest marked output is 12.40 dB

The block-size ablation, in Table 7.5, shows the cost of a finer grid. Reducing B from 8 to 4 raises pixel-level precision from 0.9599 to 0.9794 but lowers in-region PSNR from 29.63 dB to 26.78 dB, a loss of 2.85 dB. Payload falls from 128 to 32 bits per block and the tag from 32 to 16 bits, so the per-block false-accept probability rises from 2^-32 to 2^-16, an expected 0.25 false accepts per 512x512 image with 16,384 blocks, a finding and not a defect to be hidden.

Table 7.5: Block-size ablation: the resolution against payload trade-off

Block size	Pixel precision	In-region PSNR (dB)	Payload per block	Tag length
8 x 8 (main)	0.9599	29.63	128 bits	32 bits
4 x 4 (ablation)	0.9794	26.78	32 bits	16 bits

Against the six objectives of Section 2.2, O1, O2, O3, O4 and O6 are met by the measurements above. O5, recovery, is met with the rate disclosed: rho is reported at every ratio, the unrecoverable fraction is marked, and the structural limitation relative to reference-sharing schemes is stated beside the result.

7.2 CONCLUSIONS

This project built a deterministic, training-free self-embedding fragile watermarking scheme. Each 8x8 block carries, in its two least-significant bit-planes, a 32-bit keyed HMAC-SHA256 tag and a 96-bit descriptor of a spatially distant partner block chosen by a key-seeded single-cycle permutation. The implementation is 9,929

lines of Python across 24 modules, plus a local web application and demonstration interface, with no machine-learning framework or trained parameter.

Over 3,232 rows and 32 images, imperceptibility is 45.479 dB for Variant A, block-level recall is 1.0000 for three of four tamper classes, the null condition shows no false-positive block in 1,802,240 verifications, and recoverability is 3 to 6 percentage points above its structural bound. In-region recovery quality, at 29.81 dB, is modest, and whole-image figures are well below fountain-coded and reference-sharing schemes.

The claim is narrower than accuracy: the design offers accountability, since every verification decision is one recomputed keyed hash comparison that any third party holding the key can reproduce bit for bit. The distant-mapping idea is not claimed as new, nor is superiority over learned proactive watermarking. The scheme is fully fragile, protecting images only inside a lossless custody chain; its one-to-one backup allocation fails all-or-nothing per block, capping whole-image reconstruction at high tamper ratios; and security rests on key secrecy. Section 7.3 addresses these limits.

7.3 FUTURE WORK

Five directions follow from the measurements and Section 6.2.

- Reference sharing or fountain-coded descriptor allocation is the highest-value upgrade: mapping coverage, not descriptor fidelity, limits recovery quality (Section 7.1.3), so spreading each descriptor across several blocks addresses it directly.
- A semi-fragile mode tolerant of benign JPEG re-encoding would extend the scheme beyond a strictly lossless custody chain, its largest deployment restriction.
- Finer or hierarchical localization grids that do not pay the full tag cost would raise pixel-level precision beyond 0.9599 without the halved tag and false-accept penalty of $B = 4$.
- Integration into capture firmware and PACS or DICOM ingest would test the scheme where the applications of Section 6.3 apply and measure embedding cost on constrained hardware.

- A formal security proof of the keyed construction would replace the present argument from the standard properties of HMAC-SHA256 [20] with a stated reduction covering every bound field.

REFERENCES

[1] J. Fridrich and M. Goljan, "Protection of digital images using self-embedding," in Proc. Symp. Content Security and Data Hiding in Digital Media, Newark, NJ, USA, May 1999.

[2] P. Zhou, X. Han, V. I. Morariu, and L. S. Davis, "Learning rich features for image manipulation detection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, Jun. 2018.

[3] Y. Wu, W. AbdAlmageed, and P. Natarajan, "ManTra-Net: Manipulation tracing network for detection and localization of image forgeries with anomalous features," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, Jun. 2019.

[4] P. W. Wong and N. Memon, "Secret and public key image watermarking schemes for image authentication and ownership verification," IEEE Trans. Image Process., vol. 10, no. 10, pp. 1593-1601, Oct. 2001.

[5] C. Rey and J.-L. Dugelay, "A survey of watermarking algorithms for image authentication," EURASIP J. Appl. Signal Process., vol. 2002, no. 6, pp. 613-621, 2002.

[6] I. J. Cox, J. Kilian, F. T. Leighton, and T. Shamoon, "Secure spread spectrum watermarking for multimedia," IEEE Trans. Image Process., vol. 6, no. 12, pp. 1673-1687, Dec. 1997.

[7] M. Holliman and N. Memon, "Counterfeiting attacks on oblivious block-wise independent invisible watermarking schemes," IEEE Trans. Image Process., vol. 9, no. 3, pp. 432-441, Mar. 2000.

[8] C.-Y. Lin and S.-F. Chang, "Semi-fragile watermarking for authenticating JPEG visual content," in Proc. SPIE 3971, Security and Watermarking of Multimedia Contents II, 2000, pp. 140-151.

[9] J. Fridrich, "Security of fragile authentication watermarks with localization," in Proc. SPIE 4675, Security and Watermarking of Multimedia Contents IV, San Jose, CA, USA, Jan. 2002, pp. 691-700.

[10] P.-L. Lin, C.-K. Hsieh, and P.-W. Huang, "A hierarchical digital watermarking method for image tamper detection and recovery," Pattern Recognit., vol. 38, no. 12, pp. 2519-2529, Dec. 2005.

[11] X. Zhang, S. Wang, Z. Qian, and G. Feng, "Reference sharing mechanism for watermark self-embedding," IEEE Trans. Image Process., vol. 20, no. 2, pp. 485-495, Feb. 2011.

[12] P. Korus and A. Dziech, "Efficient method for content reconstruction with self-embedding," IEEE Trans. Image Process., vol. 22, no. 3, pp. 1134-1147, Mar. 2013.

[13] A. Aminuddin and F. Ernawan, "AuSR1: Authentication and self-recovery using a new image inpainting technique with LSB shifting in fragile image watermarking," J. King Saud Univ. - Comput. Inf. Sci., 2022.

[14] A. Aminuddin and F. Ernawan, "AuSR2: Image watermarking technique for authentication and self-recovery with image texture preservation," Comput. Electr. Eng., vol. 102, art. 108207, 2022.

[15] A. Aminuddin and F. Ernawan, "AuSR3: A new block mapping technique for image authentication and self-recovery to avoid the tamper coincidence problem," J. King Saud Univ. - Comput. Inf. Sci., vol. 35, no. 9, art. 101755, 2023.

[16] A. Aminuddin, F. Ernawan, D. Nincarean, A. Amrullah, and D. Ariatmanto, "TCBR and TCBD: Evaluation metrics for tamper coincidence problem in fragile image watermarking," Eng. Sci. Technol. - Int. J., vol. 56, art. 101790, 2024.

[17] X. Zhang, R. Li, J. Yu, Y. Xu, W. Li, and J. Zhang, "EditGuard: Versatile image watermarking for tamper localization and copyright protection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2024.

[18] Q. Wu, H. Li, M. Li, and M. Wang, "Multi-feature fragile image watermarking algorithm for tampering blind-detection and content self-recovery," Computers, Materials & Continua, vol. 86, no. 1, 2026.

[19] X. Zhang and S. Wang, "Fragile watermarking with error-free restoration capability," IEEE Trans. Multimedia, vol. 10, no. 8, pp. 1490-1499, Dec. 2008.

[20] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS Publication 180-4, Aug. 2015.

[21] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600-612, Apr. 2004.

[22] University of Southern California, Signal and Image Processing Institute, "The USC-SIPI Image Database." [Online]. Available: https://sipi.usc.edu/database/

[23] Eastman Kodak Company, "Kodak Lossless True Color Image Suite." [Online]. Available: https://r0k.us/graphics/kodak/

[24] Coalition for Content Provenance and Authenticity, "C2PA Technical Specification." [Online]. Available: https://c2pa.org/specifications/

[25] "A recent survey of self-embedding fragile watermarking scheme for image authentication with recovery capability," EURASIP J. Image Video Process., 2019.

[26] "Security analysis of a self-embedding fragile image watermark scheme," arXiv:1812.11735, 2018.

APPENDIX A

FEASIBILITY ASSESSMENT, SATISFIABILITY ANALYSIS AND COMPLEXITY CLASSIFICATION

A.1 FORMAL SYSTEM DEFINITION

The scheme is defined as a tuple of eleven elements, over which three functions operate. The tuple is

$$S = \{ I, Ks, ID, B, K, m, r, a, T, U, F \}$$

where I is the host image, a member of {0,...,255}^(MxN) for each colour channel; Ks is the secret key from which all key material is derived; ID is the image identifier bound into every tag; and B is the block size, 8 in the main configuration. K = (M/B) (N/B) is the number of blocks, and m is the block mapping, a permutation of {0,...,K-1} under which the descriptor of block i is stored in block m(i).

The remaining elements are derived from these. The descriptor set r = { r_i } holds one summary per block, with r_i in {0,1}^96, and the tag set a = { a_i } holds one authentication tag per block, with a_i in {0,1}^32. T is the flagged block set produced by verification, and U = { i in T : m(i) in T } is the subset of T whose descriptors are held in blocks that are themselves flagged, and which therefore cannot be rebuilt. F is the set of functions {embed, detect, recover}, with the following signatures.

- embed: (I, Ks, ID, B) -> (I', m, r, a), where I' differs from I only in its two least-significant bit-planes.
- detect: (I*, Ks, ID, B) -> T, the flagged set, with a per-block audit record, where I* is the received image.
- recover: (I*, T, m, Ks, ID) -> (I^, U), the restored image and the unrecoverable set, whose blocks are marked and not filled.

Success is stated in set terms. Let G be the ground-truth tampered block set. The system succeeds on an input when T = G and U is reported alongside the restored image. It fails when T differs from G, or when any block in T was flagged although its recomputed tag verified correctly. The 17 MSB-preserved-miss blocks of the main grid are the measured residual of the first failure mode.

A.2 SATISFIABILITY ANALYSIS

The design is treated as a constraint system, and its satisfiability is shown by exhibiting an assignment that meets every constraint, not by assertion. The constraints are as follows.

- C1, capacity: t(B) + d(B) <= 2B^2, where t is the tag length and d the descriptor length. At B = 8 this reads 32 + 96 <= 128.
- C2, bijectivity: m must be a permutation of {0,...,K-1}.
- C3, separation: ||c(i) - c(m(i))||_2 >= d_min for all i, where c(i) is the centre of block i and d_min = (1/4) min(M, N).
- C4, no fixed points and no two-cycles, which is implied by C5.
- C5, single-cycle structure: m must consist of one K-cycle.
- C6, single-pass consistency: the tag must be computable before the payload is written and must remain valid after it is written.

A satisfying assignment exists. C1 holds by the budget arithmetic, and holds with equality, so the payload is exactly capacity-tight with no spare bit. C2, C4 and C5 hold by construction, because m is read directly off a cryptographically seeded Fisher-Yates cycle order. No rejection sampling is needed, and the assignment is found in O(K) time.

C6 holds because the tag is computed over MSB(b_i) = b_i AND 0xFC, and embedding writes only the two least-significant bits. The signed quantity is therefore invariant under embedding, and the tag computed before writing is the tag verified afterward.

C3 is the only constraint that the cycle construction does not guarantee. It is enforced by a repair pass over the cycle. That pass is a local heuristic, not a global optimum, which is sufficient here because C3 is a lower bound and not an objective: any assignment that meets the bound is acceptable. The constraint system is therefore satisfiable, and a satisfying assignment is constructed in linear time for every key.

A.3 COMPLEXITY CLASSIFICATION

Embedding performs, per block, one BxB DCT, one HMAC-SHA256 evaluation and a fixed number of bit operations. Verification performs the same work, and the mapping costs O(K). With K = MN/B^2 blocks, the total running time is

$$T(M, N) = O(MN)$$

with no iteration, no optimization and no training, and memory is O(MN + K). The decision problem "does block i verify under key Ks" is answered by one deterministic hash computation and one comparison, and it is therefore in P. No NP-hard or NP-complete sub-problem arises anywhere in the pipeline. The measured timings, 473 ms to embed and 197 ms to verify a 512x512 colour image on a single CPU thread, are consistent with this linear classification.

A related problem is combinatorially hard, however, and the report should not hide it. Choosing a descriptor allocation that maximizes expected recoverability over an unknown tamper geometry is equivalent to choosing a permutation that maximizes the expected size of the set of flagged blocks whose partners survive. This is an assignment problem over K! candidate permutations, closely related to quadratic-assignment formulations that are NP-hard in general.

This project does not solve that optimization. It sidesteps it with a constructive permutation that satisfies a separation lower bound, and it reports the resulting recoverability rate as a measurement, without claiming optimality. Reference-sharing and fountain-coded allocations [11], [12] change the structure of this problem rather than solving it, which is why they are listed as future work.

A.4 FEASIBILITY ASSESSMENT

A.4.1 Technical Feasibility

The pipeline is implemented and measured. It runs on a commodity CPU, embedding a 512x512 colour image in 473 ms and verifying it in 197 ms with no GPU, and it uses only standard-library cryptography. The complete evaluation grid, 3,232 rows over 32 images, has already been produced, so every claim in this report rests on measurement and not on projection.

A.4.2 Economic Feasibility

No licensed software, GPU, cloud compute or paid dataset is required. The corpus is public and pinned by SHA-256 hash, so it reproduces byte for byte. The direct cost is student time and a commodity workstation, and the software stack is entirely open source.

A.4.3 Operational Feasibility

Verification requires only the received image, the key and the image identifier. The operational burden is therefore key management, which is the one genuine operational precondition of the scheme: an adversary holding the key can re-embed, so custody of the key is custody of the evidence.

A.4.4 Schedule Feasibility

The experimental work was completed before this report was written, which removes the usual end-of-project measurement risk. The remaining work is the reference-sharing extension and the final submission, neither of which alters the measured baseline reported here.

APPENDIX B
DETAILS OF THE PAPERS REFERRED

[1] J. Fridrich and M. Goljan, "Protection of digital images using self-embedding," in Proc. Symp. Content Security and Data Hiding in Digital Media, Newark, NJ, USA, May 1999.

Summary: The paper introduces self-embedding: a reduced-quality version of the image is hidden in the image's own least significant bits, so that a cropped or replaced region can be approximately restored from the surviving area. It is the origin of the recovery line of work, and this project inherits the idea that an image carries its own repair data.

[2] P. Zhou, X. Han, V. I. Morariu, and L. S. Davis, "Learning rich features for image manipulation detection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, Jun. 2018.

Summary: A two-stream network combines an RGB stream, which looks for visible tampering artifacts, with a noise stream, which looks for inconsistencies in local noise statistics, to detect and localize manipulated regions. It represents passive detection, which analyzes the received image alone. This project cites it as the contrast case: detection without any embedded reference, and without any recovery.

[3] Y. Wu, W. AbdAlmageed, and P. Natarajan, "ManTra-Net: Manipulation tracing network for detection and localization of image forgeries with anomalous features," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, Jun. 2019.

Summary: ManTra-Net learns features that describe how locally anomalous each pixel is relative to its surroundings, which lets one network handle many forgery types and localize them at pixel level. Like [2], it is a passive detector that cannot restore content. It is cited to show why a tamper-localization result alone does not answer the recovery requirement of this project.

[4] P. W. Wong and N. Memon, "Secret and public key image watermarking schemes for image authentication and ownership verification," IEEE Trans. Image Process., vol. 10, no. 10, pp. 1593-1601, Oct. 2001.

Summary: The scheme computes a keyed cryptographic hash over each image block, together with block-specific information, and hides the result in the least significant bits of that same block, giving block-level localization of tampering. It established the block-wise fragile authentication template. This project follows the block-and-tag structure, and its tag construction answers the weaknesses later shown against it.

[5] C. Rey and J.-L. Dugelay, "A survey of watermarking algorithms for image authentication," EURASIP J. Appl. Signal Process., vol. 2002, no. 6, pp. 613-621, 2002.

Summary: The survey classifies authentication watermarks by what they tolerate, separating fragile schemes that flag any change from semi-fragile schemes that survive acceptable processing such as compression, and it lists the requirements of localization and security. It supplied the taxonomy used to place this project among fragile schemes, and to state why compression tolerance is deliberately not claimed.

[6] I. J. Cox, J. Kilian, F. T. Leighton, and T. Shamoon, "Secure spread spectrum watermarking for multimedia," IEEE Trans. Image Process., vol. 6, no. 12, pp. 1673-1687, Dec. 1997.

Summary: A watermark is spread across the perceptually significant frequency components of the image, which makes it hard to remove without visibly damaging the content. This is the robust watermarking tradition, designed to survive processing for ownership claims. It is cited as the opposite design goal: this project wants a mark that breaks on any alteration, not one that persists.

[7] M. Holliman and N. Memon, "Counterfeiting attacks on oblivious block-wise independent invisible watermarking schemes," IEEE Trans. Image Process., vol. 9, no. 3, pp. 432-441, Mar. 2000.

Summary: When each block is marked independently of its position and of the rest of the image, an attacker holding several marked images can assemble a

forgery from valid blocks (the vector quantization or collage attack). The paper shows that independence between blocks is the vulnerability. It forces the tag in this project to bind the block index and an image identifier.

[8] C.-Y. Lin and S.-F. Chang, "Semi-fragile watermarking for authenticating JPEG visual content," in Proc. SPIE 3971, Security and Watermarking of Multimedia Contents II, 2000, pp. 140-151.

Summary: Authentication bits are derived from relationships between quantized DCT coefficients that survive JPEG recompression, so the mark rejects malicious edits while accepting lossy compression. It is the standard example of semi-fragile design. This project sits on the strict side of that trade: it reports compression as a tamper, and does not claim the tolerance this paper achieves.

[9] J. Fridrich, "Security of fragile authentication watermarks with localization," in Proc. SPIE 4675, Security and Watermarking of Multimedia Contents IV, San Jose, CA, USA, Jan. 2002, pp. 691-700.

Summary: The paper analyzes fragile localizing schemes and shows that the finer the localization, the more easily block-level independence can be exploited, so small blocks and strong security pull against each other. It argues for dependence between blocks and across the image. This project cites it for the security argument behind tying each tag to other blocks and to an image identifier.

[10] P.-L. Lin, C.-K. Hsieh, and P.-W. Huang, "A hierarchical digital watermarking method for image tamper detection and recovery," Pattern Recognit., vol. 38, no. 12, pp. 2519-2529, Dec. 2005.

Summary: Detection is arranged in levels, from larger regions down to small blocks, so that a block missed at one level can be caught at the next, and recovery data for each block is stored in a different, mapped block. The hierarchy reduces false negatives. This project cites it as an early combined detection-and-recovery scheme and as a source of the mapped-storage practice.

[11] X. Zhang, S. Wang, Z. Qian, and G. Feng, "Reference sharing mechanism for watermark self-embedding," IEEE Trans. Image Process., vol. 20, no. 2, pp. 485-495, Feb. 2011.

Summary: Reference data are shared among blocks through a redundancy-style mechanism, so recovery degrades gradually as the tampered fraction grows, and quality stays acceptable when the damage is large. It is one of the two works that define the graceful-degradation benchmark. This project does not yet meet that benchmark, and the report states the gap plainly.

[12] P. Korus and A. Dziech, "Efficient method for content reconstruction with self-embedding," IEEE Trans. Image Process., vol. 22, no. 3, pp. 1134-1147, Mar. 2013.

Summary: Reference information is split into several shares and embedded so that content can be reconstructed from the pieces that survive, and quality falls smoothly as more of the image is destroyed. Together with [11], it sets the graceful-degradation benchmark for self-embedding. This project falls short of that benchmark, and the report treats it as the main direction for future work.

[13] A. Aminuddin and F. Ernawan, "AuSR1: Authentication and self-recovery using a new image inpainting technique with LSB shifting in fragile image watermarking," J. King Saud Univ. - Comput. Inf. Sci., 2022.

Summary: The paper stores recovery information with a shift of the least significant bits, which gives a larger payload at tolerable distortion, and it fills unrecovered pixels by image inpainting. It is the first of the three AuSR schemes. This project borrows the LSB-shifting idea, and credits it to this paper.

[14] A. Aminuddin and F. Ernawan, "AuSR2: Image watermarking technique for authentication and self-recovery with image texture preservation," Comput. Electr. Eng., vol. 102, art. 108207, 2022.

Summary: The second AuSR scheme adjusts the embedding so that image texture is preserved, because heavy bit modification shows up most in textured areas. It is a refinement of how watermark bits are placed, not a new recovery principle. This project cites it as the closest reference for the imperceptibility side of its design.

[15] A. Aminuddin and F. Ernawan, "AuSR3: A new block mapping technique for image authentication and self-recovery to avoid the tamper coincidence

problem," J. King Saud Univ. - Comput. Inf. Sci., vol. 35, no. 9, art. 101755, 2023.

Summary: Tamper coincidence arises when a block and the block holding its recovery data are both altered, so neither can be restored. AuSR3 proposes a block mapping that places the recovery data far from the block it describes. This project credits the distant-mapping principle to this paper and does not claim it.

[16] A. Aminuddin, F. Ernawan, D. Nincarean, A. Amrullah, and D. Ariatmanto, "TCBR and TCBD: Evaluation metrics for tamper coincidence problem in fragile image watermarking," Eng. Sci. Technol. - Int. J., vol. 56, art. 101790, 2024.

Summary: The paper defines two metrics, TCBR and TCBD, that measure how much a scheme suffers from tamper coincidence, so that mapping designs can be compared on a common scale instead of by example images. This project uses it as the reference point for reporting coincidence behavior, and for stating which parts of that measurement it adopts.

[17] X. Zhang, R. Li, J. Yu, Y. Xu, W. Li, and J. Zhang, "EditGuard: Versatile image watermarking for tamper localization and copyright protection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2024.

Summary: EditGuard uses a learned encoder and decoder to embed both a localization mark and a copyright message, aimed at edits made by generative models. It marks the move of watermarking toward learned, AI-era settings. This project cites it as the current learned alternative, and keeps to a classical scheme whose behavior can be inspected and verified.

[18] Q. Wu, H. Li, M. Li, and M. Wang, "Multi-feature fragile image watermarking algorithm for tampering blind-detection and content self-recovery," Computers, Materials & Continua, vol. 86, no. 1, 2026.

Summary: The scheme builds the watermark from several image features and detects tampering blindly, without access to the original, while storing information for content self-recovery. It is among the most recent entries in the line of fragile self-recovery work. This project cites it to show that the problem is still active, and it does not compare against it directly.

[19] X. Zhang and S. Wang, "Fragile watermarking with error-free restoration capability," IEEE Trans. Multimedia, vol. 10, no. 8, pp. 1490-1499, Dec. 2008.

Summary: The scheme embeds the original values of the pixels it modifies, together with authentication data, so that an untampered image can be restored exactly to its original state after verification. It targets error-free restoration of the cover image, not recovery of tampered content. This project cites it to separate those two meanings of restoration.

[20] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS Publication 180-4, Aug. 2015.

Summary: The standard specifies the SHA family of hash functions, including SHA-256, which maps input of any length to a fixed-length digest that is hard to invert or to collide. It is the primitive underneath the keyed tags in this project. The tag construction relies on SHA-256 as specified here, and it does not invent a hash.

[21] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600-612, Apr. 2004.

Summary: The paper proposes the structural similarity index, which compares luminance, contrast and structure between two images and tracks perceived quality better than error energy alone. This project uses SSIM, alongside PSNR, to report how much the embedding changes the image and how good a recovered image is.

[22] University of Southern California, Signal and Image Processing Institute, "The USC-SIPI Image Database." [Online]. Available: https://sipi.usc.edu/database/

Summary: The database is a long-standing public collection of digitized test images, widely used as a common benchmark in image processing research. It is cited as a dataset source for the evaluation images, so that results can be compared with those in the published literature.

[23] Eastman Kodak Company, "Kodak Lossless True Color Image Suite." [Online]. Available: https://r0k.us/graphics/kodak/

Summary: The suite is a set of uncompressed color photographs released for use in image compression and processing research, and it is a common benchmark for natural color images. It is cited as the second dataset source for evaluation, so that the test material is not limited to one collection.

[24] Coalition for Content Provenance and Authenticity, "C2PA Technical Specification." [Online]. Available: https://c2pa.org/specifications/

Summary: The specification defines signed provenance metadata, called content credentials, that is attached to a media file to record its origin and edit history. The credentials sit beside the pixels and can be stripped from the file. This project cites it as the metadata-based alternative, and contrasts it with a mark carried inside the pixels.

[25] "A recent survey of self-embedding fragile watermarking scheme for image authentication with recovery capability," EURASIP J. Image Video Process., 2019.

Summary: The survey reviews self-embedding fragile schemes that authenticate an image and recover tampered regions, and organizes them by how recovery data are generated, mapped and embedded. It is the broad map of the field. This project cites it for the classification of schemes and for the open problems it identifies.

[26] "Security analysis of a self-embedding fragile image watermark scheme," arXiv:1812.11735, 2018.

Summary: The paper examines a published self-embedding fragile scheme from an attacker's side and shows that its security claims do not survive analysis. It illustrates that recovery schemes are attacked in practice, not just in theory. This project cites it as supporting evidence for treating the security of the tag construction as a design requirement.

APPENDIX C
PLAGIARISM REPORT

The plagiarism check report generated by the institute's similarity checking service is to be attached at this point, as required by clause 19 of the departmental report format. The report covers Chapters 1 to 7 and Appendices A and B of this document; the cover page, certificate, acknowledgement, table of contents, lists and references are excluded from the check in line with the usual convention.

[ATTACH THE PLAGIARISM CHECK REPORT HERE]